阿里云天池 Python 训练营 - Part1

- 2022 年 02 月 14 日版本，初始版本
- 2024 年 01 月 14 日版本，修改版本
- 本文档是以 CC 开源的模式的发布，你能且将获得本文档的 PDF 版本以及 Jupyter Notebook 版本
- 本文档并不申明自己的版权信息，为了更好的知识传播，我们授权你使用本文档，你可以使用它，进行二次创作，进行分发，进行修改，并可以以此为蓝本进行授课。
- 请保留本文档的原始来源。
- 本文档首先在国际人学校组织的"阿里云天池 Python 训练营课程"中使用。
- 本文档由林肯老师首次组织和编辑。你可以通过网址 数据大咖 找到他。

- 阿里云天池 Python 训练营 - Part1
  - 1. 术语表、中英文对照、注意要点
    - 1.1 为什么要学习编程
      - 1.1.1 什么是程序
      - 1.1.2 什么是语言， 形式语言和自然语言
    - 1.2 术语表、中英文对照
    - 1.3 注意要点
  - 2. 输出
    - 2.1 基本用法
    - 2.2 格式化字符串
    - 2.3 自定义分隔符
    - 2.4 禁止自动换行
    - 2.5 输出到文件
    - 2.6 转义字符
    - 2.7 输出 Unicode 字符
    - 2.8 调试时输出变量名和值
    - 2.9 使用 `print` 输出复杂数据结构
    - 2.10 彩色输出
    - 2.11 输出带时间戳的日志
  - 3. 输入
  - 4. Python 的代码注释
    - 4.1 单行注释
    - 4.2 多行注释
  - 5. Python 对象详解

- 5.1 类的定义(defination)
- 5.2 对象的身份(id)
- 5.3 对象的类型(type)
- 5.4 对象的值(value)
- 5.5 对象的属性(attribute)
- 5.6 对象的方法(function)
- 5.7 对象与变量
- 5.8 Python 中的数字与字符串
  - 5.8.1 数字
  - 5.8.2 字符串
- 5.9 算术运算符
  - 5.9.1 加(+)
  - 5.9.2 减(-)
  - 5.9.3 乘(*)
  - 5.9.4 除(/)
  - 5.9.5 取模(%)
  - 5.9.6 幂(**)
  - 5.9.7 取整数(//)
- 5.10 比较运算符
  - 5.10.1 等于(==)
  - 5.10.2 不等于(!=)
  - 5.10.3 大于(>)
  - 5.10.4 小于(<)
  - 5.10.5 大于或等于(>=)
  - 5.10.6 小于或等于(<=)
- 5.11 赋值运算符
  - 5.11.1 赋值运算
  - 5.11.2 累积式赋值运算
- 5.12 逻辑运算符
  - 5.12.1 and（与）
  - 5.12.2 or（或）
  - 5.12.3 not（非）
- 5.13 成员运算符
- 5.14 格式化字符串
  - 5.14.1 使用位置和关键字格式化字符串
  - 5.14.2 数字格式设置

        - 5.14.3 对齐设置
    - 5.15 pythontutor 使用
- 6. Python 模块、包、库，三方库安装
    - 6.1 什么是模块、包、库
    - 6.2 Excel 读取库 xlrd
        - 6.2.1 xlrd 模块导入
        - 6.2.2 读取 Excel 工作表
        - 6.2.3 读取 Excel 行、列、单元格信息
    - 6.3 Excel 写入库 xlwt
        - 6.3.1 新建工作簿、新建工作表和写入数据
    - 6.4 Excel 修改库 xlutils
- 7. 条件语句
    - 7.1 if 语句
        - 7.1.1【例子】if 标准用法
        - 7.1.2【例子】if 标准用法
        - 7.1.3【综合案例】根据分数判断等级
    - 7.2 if - else 分支语句
        - 7.2.1【例子】猜一猜小姐姐想的是哪个数字?
        - 7.2.2【例子】`else` 的缩进问题
        - 7.2.3【例子】猜一猜小姐姐想的是哪个数字?
    - 7.3 if - elif - else 分支语句
    - 7.4 if 条件分支语句单行写法
    - 7.5 assert 关键词
    - 7.6 案例：对数字进行分类计数
- 8. 循环语句
    - 8.1 while 循环
        - 8.1.1 【例子】猜一猜小姐姐想的是哪个数字?
        - 8.1.2【例子】循环字符串
        - 8.1.3【例子】布尔表达式返回 0，循环终止
    - 8.2 while - else 循环
        - 8.2.1【例子】比 5 大，还是比 5 小
        - 8.2.2【例子】增加一个 break 会怎么样
    - 8.3 【综合训练】使用 while 循环批量创建 Excel
    - 8.4 for 循环
        - 8.4.1 【例子】循环字符串
        - 8.4.2【例子】循环列表（直接法和通过长度）
        - 8.4.3【例子】循环访问字典（使用 items）

        - 8.4.4【例子】循环访问字典（使用 keys）
        - 8.4.5【例子】循环访问字典（使用 values）
        - 8.4.6【例子】循环序列数（range）
        - 8.4.7【例子】循环批量创建 Excel 文件
    - 8.5 for 循环嵌套语句
    - 8.6 【综合训练】for 使用嵌套循环输出九九乘法表并保存在 Excel 中
    - 8.7 for - else 循环
    - 8.8 enumerate()函数
    - 8.9 break 语句
    - 8.10 continue 语句
- 9. pass 语句
- 10. 推导式
    - 10.1 列表推导式
    - 10.2 元组推导式
    - 10.3 字典推导式
    - 10.4 集合推导式
- 其它
- 11. 异常处理
    - 11.1 Python 标准异常总结
    - 11.2 Python 标准警告总结
    - 11.3 try - except 语句
    - 11.4 try - except - finally 语句
    - 11.5 try - except - else 语句
    - 11.6 raise 语句

1. 术语表、中英文对照、注意要点



人类语言是如何产生的

1.1 为什么要学习编程

本次课程的目标是教会你像计算机科学家一样思考。

这种思考方式综合了数学、工程学以及自然科学的一些最优秀的特性。计算机科学家与数学家类似，他们使用形式语言来描述理念(特别是计算）；与工程师类似，他们设计产品，将元件组装成系统，对不同的方案进行评估选择；与自然科学家类似，他们观察复杂系统的行为，构建科学假说，并检验其预测。

作为计算机科学家，最重要的技能就是**问题求解**。

问题求解是发现问题、创造性地思考解决方案以及清晰准确地表达解决方案的能力。

实践证明，学习编程的过程，正是训练问题求解能力的绝佳机会。

一方面，你将学会编程，其本身就是一个非常有用的技能；

另一方面，你可以使用编程作为工具，去达到更高的目标。

1.1.1 什么是程序

程序是指一组定义如何进行计算的指令的集合。这种计算可能是数学计算，如解方程组或者查找多项式的根，也可以是符号运算，如搜索和替换文档中的文本，或者图形相关的操作，如处理图像或播放视频。

在不同的编程语言中，程序的细节有所不同，但几乎所有编程语言中都会出现以下几类基本指令。

- 输入(input)：从键盘、文件或者其他设备中获取数据。
- 输出(output)：将数据显示到屏幕，保存到文件中，或者发送到网络上等。
- 数学(math)：进行基本数学操作，如加法或乘法。

- 条件执行(if)：检查某种条件的状态，并执行相应的代码。
- 重复(for,while)：重复执行某种动作，往往在重复中有一些变化。

信不信由你，这差不多就是全部了😀。

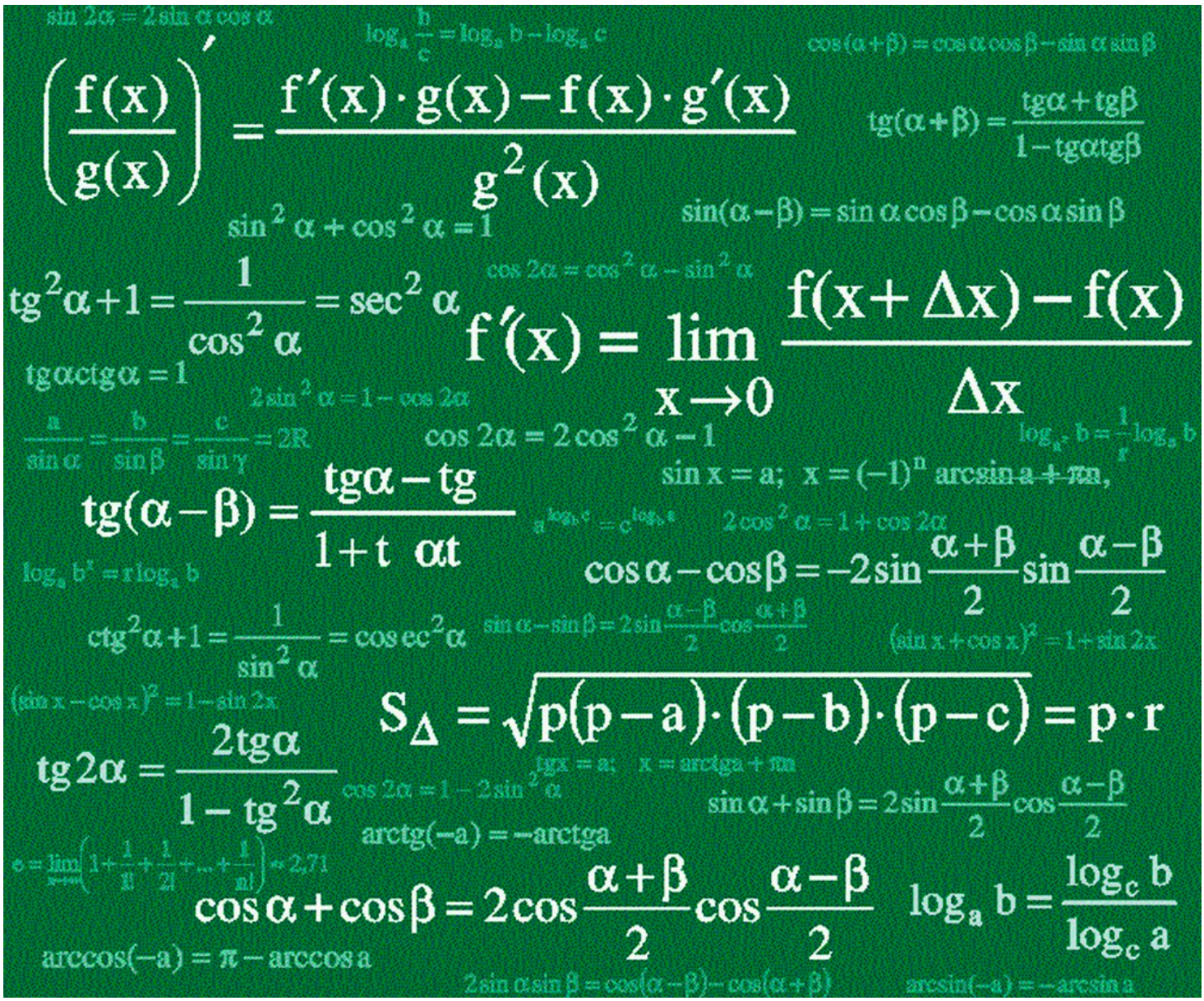
你所遇到过的所有程序，无论多么复杂，都是由类似上面的这些指令组成的。

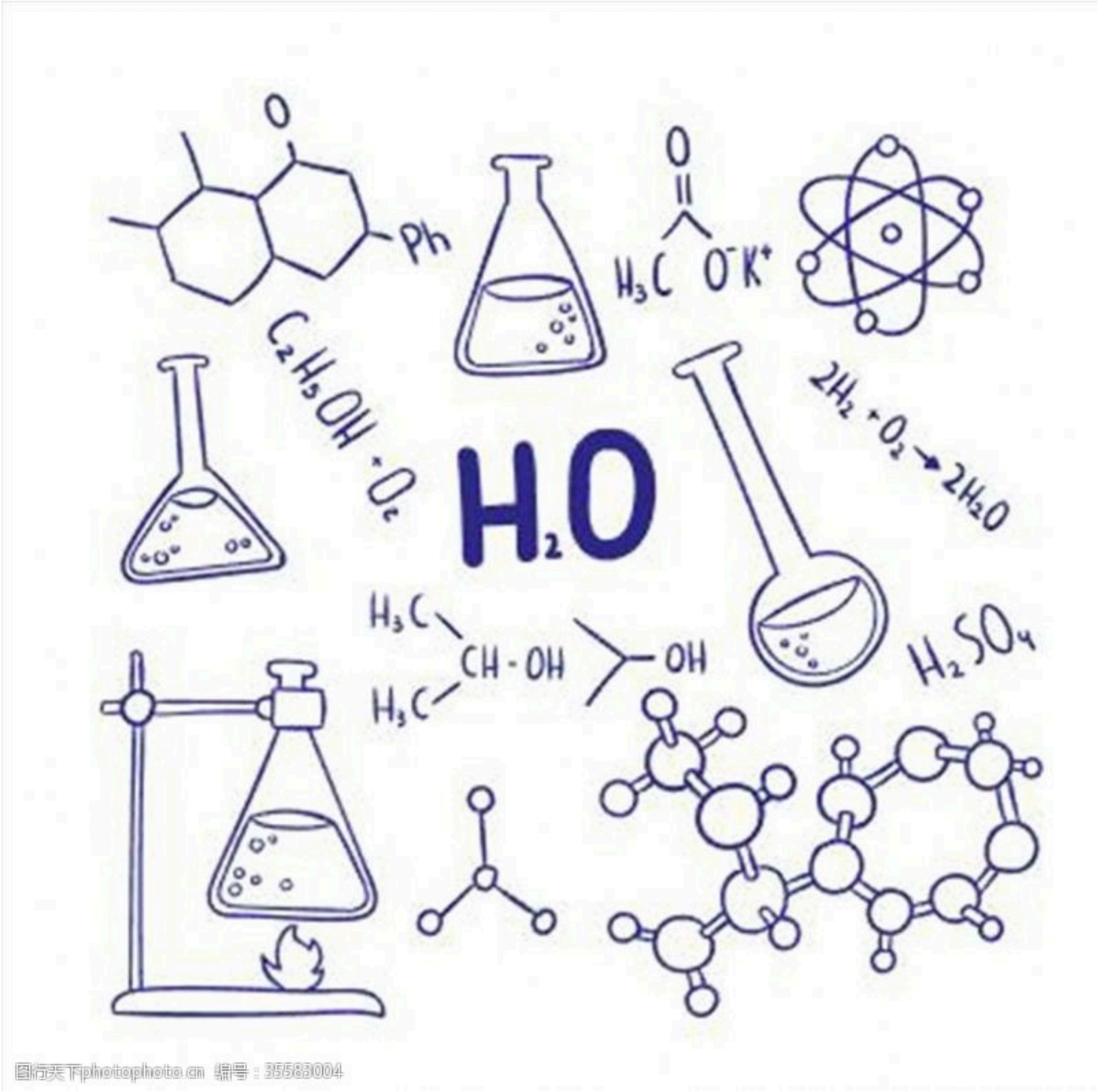
所以我们可以把编程看作一个将大而复杂的任务分解为更小的子任务的过程，不断分解，直到任务简单到足以由上面的这些基本指令组合完成。

1.1.2 什么是语言， 形式语言和自然语言

自然语言是指人们所说的语言，如英语、西班牙语和法语。它们不是由人设计而来的(虽然人们会尝试加以语法限制），而是自然演化而来的。

形式语言 则是人们为了特殊用途设计的语言。





例如，数学上使用的符号体系是一种特别擅于表示数字和符号之间关系的形式语言；化学家则使用另一种形式语言来表示分子的化学结构。

而最重要的是：**编程语言是人们为了表达计算过程而设计出来的形式语言。**

形式语言倾向于对语法做出严格的限制。例如，`3+3=6` 是语法正确的数学表达式，但 `3+=3$6` 则不是。

H_2O 是语法正确的化学方程式，而 2Zz 则不是。

语法规则有两种，分别适用于记号(token）和结构(structure）。

记号是语言的基本元素，如词、数字和化学元素。

3+=3$6 的一个问题就是 **$** 在数学表达式中(至少就我所知）不是合法记号。相似地，2Zz 不合法是因为并不存在缩写为 Zz 的化学元素。

第二种语法规则指定记号所组合的方式。

数学等式 **3+=3** 不合法，因为虽然 `+` 和 `=` 是合法记号，但不能将它们连续放置。

相似地，在化学表达式里，下标数字应该出现在元素名称之后，而不是之前。

"This is@well-structured EngliSh sentence with invalid t*kens in it." 是一个结构良好，但包含非法记号的英语语句。

"This sentence all valid tokens has，but invalid structure with." 这句话所有的记号都合法，但是语句结构不合法。

当你阅读英语的句子或形式语言的语句时，需要弄清句子的结构是什么(虽然在自然语言中这个过程是下意识完成的）。这个过程称为语法分析。

虽然形式语言和自然语言有很多共同的特点－**记号、结构、语法以及语义**，但它们也有一些区别。

- 歧义性：
    - 自然语言充满了歧义，人们通过上下文线索和其他信息来处理这些歧义。
    - 形式语言通常设计为几乎或者完全没有歧义，即不论上下文环境如何，任何表达式都只有一个含义。
- 冗余性：
    - 为了弥补歧义，减少误解，自然语言采用大量的冗余。因此，自然语言往往很啰唆。形式语言则相对不那么冗余，更加简洁。
- 字面性：
    - 自然语言充满了习惯用语和比喻。例如，有人说，"硬币掉了"(The penny dropped），并不一定是硬币，也不一定是有什么掉了。
    - 形式语言则严格按照它的字面意思表达含义。

形式语言的密度远远大于自然语言，所以阅读起来需要花费更多的时间。还有，结构非常重要，所以直接自顶向下、从左至右的阅读顺序并不一定是最好的。相反，要试着学会在头脑中解析程序，辨别出记号并解析出结构。最后，细节很重要。在自然语言中常常可以忽略的小错误，如拼写错误或者标点符号错误，在形式语言中往往会造成很大的差别。

1.2 术语表、中英文对照

#	英文	中文

1	byte	字节
2	int	整型、整数
3	float	浮点型、小数
4	complex	复数
5	string	字符串
6	char	字符
7	tuple	元组类型
8	list	列表类型
9	dict	字典类型
10	set	集合类型
11	date	日期类型
12	time	时间类型
13	random	随机数
14	row	行
15	column	列
16	table	表格
17	range	范围
18	if	判断(如果)
19	else	其他
20	elif	其他分支(判断)
21	for	循环
22	while	循环
23	continue	跳出循环，继续下一次
24	break	跳出整个循环
25	add,addition	加法
26	sub,subtraction	减法
27	mul,multiple,multiplication	乘法

28	div,division	除法
29	function	函数
30	method	方法
31	return	返回，返回值
32	result	结果
33	def,defination,define	定义
34	max	最大
35	min	最小
35	avg	平均值
36	parameter	参数
37	info,infomation	信息
38	lambda	匿名函数
39	iterator	迭代器
40	generator	生成器
41	True	真，真值
42	False	假，假值
43	next	下一个
44	OOP，Object Oriented Programming	面向对象编程
45	assert	断言
46	class	类
47	del,delete	删除
48	pop	弹出，栈推出
49	try ... except	抓住异常
50	finally	最终
51	global	全局
52	import	引入
53	or	或者

54	not	取反
55	and	并且
56	raise	抛出异常
57	with	获取句柄
58	indent,indentation	缩进
59	unindent	未缩进
60	match	匹配
61	sys,system	系统
62	std,standard	标准的
63	write	写入
64	read	读取
65	path	路径
66	dir,directory	文件夹
67	file	文件
68	instance	实例
69	sub	子
70	reverse	反转，abc 转成 cba
71	eval	计算有效表达式，并返回对象
72	order	排序
73	trace	堆栈
74	call	调用
75	support	支持
76	unsupported	不支持的
77	in	包含
78	not in	未包含
79	is	是某对象
80	is not	不是某对象

81	center	居中
82	count	计数，计算出现次数
83	encode	编码
84	decode	解码
85	endswith	以... 结尾
86	find	查找
87	index	索引位置
88	isnumeric	是否是数字类型
89	isalpha	是否是字符
90	isalnum	是否是字符或数字
91	islower	是否是小写
92	isupper	是否是大写
93	isspace	是否是空格
94	join	合并字符串
95	len,length	字符串长度
96	strip	截掉空格或指定字符
97	replace	替换
98	split	以... 为分隔符截取字符串
99	startswith	是否以... 开头
100	append	追加
101	insert	插入
102	remove	移除
103	sort	排序
104	clear	清空列表
105	copy	复制...
106	items	获取... 中的项
107	keys	获取... 中的键

108	values	获取... 中的值
109	difference,diff	多个集合的差集
110	intersection	多个集合的交集
111	subset	子集
112	superset	超集
113	union	并集
114	symmetric_difference	获取两个结合中不重复的元素集合
115	pass	过
116	module	模块
117	package	包
118	application	应用程序
119	filter	过滤器
120	mkdir	创建目录
121	rename	重命名
122	unlink	删除文件路径
123	error	错误
124	warn	警告
125	init	初始化
126	public	公共的
127	private	私有的
128	protected	受保护的
129	mod	求余数
130	pow	乘方
131	cmp,compare	比较
132	namespace	命名空间
133	math	数学包
134	sin	正弦

135	cos	余弦
136	log	对数
137	request	网络请求包
138	response	响应
139	regex	正则表达式

1.3 注意要点

#	符号	中文符号	英文符号
1	单引号	‘’	''
2	双引号	“”	""
3	小括号	（）	()
4	感叹号	！	!
5	百分号	%	%
6	中括号	【】	[]
7	大括号	{}	{}
8	问号	？	?
9	分号	；	;
10	斜杆	/	/
11	冒号	：	:
12	货币	￥	$
13	井号	#	#
14	反斜杠	\	\
15	竖杠	|	|
16	加号	＋	+
17	减号	－	-
18	星号	＊	*
19	大于	＞	>

20	小于	<	<
21	等号	=	=
22	大于等于	> =	>=
23	小于等于	< =	<=

2. 输出

一般查看输出结果可以使用 `print` 函数

```
print('嗨! python 我来了!')
```

以下是 Python 中 `print()` 函数的一些常见用法和示例，涵盖了基础到高级的使用场景：

2.1 基本用法

输出字符串、数字、变量等。

```
# 输出字符串
print("Hello, World!")

# 输出数字
print(42)

# 输出变量
name = "Alice"
age = 25
print("Name:", name, "Age:", age)
```

2.2 格式化字符串

使用 `f-string`、`str.format()` 和 `%` 格式化输出。

```
# f-string
name = "Bob"
score = 95.6
print(f"{name} scored {score:.1f} points.")

# str.format()
print("{} scored {:.1f} points.".format(name, score))
```

```
8
9  # % 格式化
10 print("%s scored %.1f points." % (name, score))
```

2.3 自定义分隔符

通过 `sep` 参数指定多个值之间的分隔符。

```
1 print("apple", "banana", "cherry", sep=", ")
```

2.4 禁止自动换行

通过 `end` 参数更改默认换行行为。

```
1 # 默认换行
2 print("Hello")
3 print("World")
4
5 # 禁止换行
6 print("Hello", end=" ")
7 print("World")
```

2.5 输出到文件

将内容写入文件而不是控制台。

```
1 with open("output.txt", "w") as file:
2     print("This will be written to a file.", file=file)
```

2.6 转义字符

在字符串中使用特殊字符。

```
1 # 换行和制表符
2 print("Hello\nWorld")  # 换行
3 print("Name\tAge")     # 制表符
4
  # 转义引号
```

```
print("He said, \"Python is awesome!\"")

```

2.7 输出 Unicode 字符

支持输出表情符号或特殊字符。

```
print("Emoji: 😊, Symbol: Ω, Chinese: 你好")
```

2.8 调试时输出变量名和值

在调试时输出变量名和值，可以结合 `f-string` 使用。

```
x = 42
y = 3.14
print(f"x = {x}, y = {y}")
```

2.9 使用 `print` 输出复杂数据结构

输出列表、字典等。

```
my_list = [1, 2, 3]
my_dict = {"name": "Alice", "age": 25}

print("List:", my_list)
print("Dictionary:", my_dict)
```

2.10 彩色输出

利用 `colorama` 或 ANSI 转义序列实现彩色输出。

```
# 使用 ANSI 转义序列
print("\033[31mThis is red text.\033[0m")
print("\033[32mThis is green text.\033[0m")

# 使用 colorama 库
from colorama import Fore, Style
print(Fore.RED + "This is red text." + Style.RESET_ALL)
print(Fore.GREEN + "This is green text." + Style.RESET_ALL)
```

```
8
```

2.11 输出带时间戳的日志

结合 `datetime` 模块输出带时间的消息。

```python
from datetime import datetime

current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
print(f"[{current_time}] Process started...")
```

3. 输入

使用 `print` 函数可以完成数据的输出，那么如何编写输入语句呢？一般使用 `input` 函数。

```python
name = input('请输入你的姓名: ')
print(name)
```

4. Python 的代码注释

代码注释就是为写好的代码片段添加注解。做代码注释有以下两好处：

- 能更好地维护项目，也能让阅读者更快地读懂代码的意思；
- 在做代码调试时，如果需要让一部分代码暂时不运行，也可以使用注释的方法。

4.1 单行注释

单行注释，即注释只作用于一行。在单独的行写注释内容之前，要输入 `#` (井号）。

```python
# 将数据输出到屏幕上
print('嗨! python 我来了! ')
# 在此输入姓名
name = input('请输入你的姓名:')
# 输出输入的数据
print(name)
```

为了减少代码行数，可以将单行注释写到每一行的后面

```python
print('嗨! python 我来了! ') #将数据输出到屏幕上
name = input('请输入你的姓名: ') # 在此输入姓名
print(name) # 将打印出输入的数据**
```

4.2 多行注释

如果有大段的注释文字要写，则可以使用多行注释的方法。多行注释的内容要包含在一对单引号中，6 个单引号为一对。**单引号中的内容不会被运行。**

```
'''
print 函数表示将数据输出
input 函数表示要接收输入的数据
两个函数可以结合使用
'''
name = input('请输入你的姓名: ')
print(name)
```

除使用单引号做注释外，也可以使用双引号来做注释。**双引号中的内容也不会被运行。**

```
"""
print 函数表示将数据输出
input 函数表示要接收输入的数据
两个函数可以结合使用
"""
name = input('请输入你的姓名: ')
print(name)
```

> 到底是使用单引号做注释，还是使用双引号做注释，没有强制规定，完全根据用户的习惯而定。

5. Python 对象详解

真实的世界是由千千万万的对象组成的。

在 Python 的编程世界里，所有的一切也可以看作对象，比如数字、字符，以及后面将会学到的列表、元组、集合、字典、函数等。

用户可以使用这些对象，也可以在 Python 中创建自己的对象。

5.1 类的定义(defination)

类也是一种对象，只不过它是用来创建对象的一种对象。类用来描述具有相同属性和方法的对象集合，它定义了该集合中每个对象所共有的属性和方法，对象是类的实例。也就是说，对象是由类创建的。比如，后面的章节中会讲解通过 list 类来创建或转换一个列表对象。

5.2 对象的身份(id)

在现实生活中，人就是一个类，而每一个具体的人就是对象，具体的人可以靠身份证号来进行识别，也可以定位所在位置。

Python 中的对象也是有身份的，可以通过 id 函数来识别对象在内存中的地址。比如，字符串 **'杨辉'** 就是对象，输入代码

```
print(id('杨辉'))
```

屏幕上的输出结果为 **2673213256496**，这串数字就可以看作该字符串在内存中的地址，并且具有唯一性。但是，这串数字是变化的，因此在测试代码时，每次输出的结果可能不一样，读者不要为此感到困惑。

5.3 对象的类型(type)

虽然万物皆对象，但对象也有类型之分。

比如，猪、狗、牛、马、花、草、树、木等都是对象，但它们却是不同的类型，猪、狗、牛、马是动物类型，花、草、树、木是植物类型。

在 Python 中，99、888、 'abc' 都是对象，9 和 888 是数字类型，而 'abc' 是字符串类型。

不同类型的对象有着不同的属性和方法，遵循不同的规则。

要查看对象的类型，可以使用 `type` 函数。

比如，输入代码 `print(type(99))`，返回结果为 `<class 'int' >`，表示 99 是 int 类型，也就是整数类型。再比如，输入代码 `print(type('abc'))`，返回结果为 `<class 'str' >`，表示 'abc' 是 str 类型，也就是字符串类型。

其他对象的类型就不一一列举了，后文中会涉及。

5.4 对象的值(value)

对象除有身份、类型外，还有值。

人的名字就可以看作值。在 Python 中，有的对象的值是可以改变的，有的对象的值不可以改变。比如，数字、字符、元组都是不可以改变值的对象。

5.5 对象的属性(attribute)

对象的特征也称为属性。比如，字符串 **'abcd'**，它的字符长度为 **4**，这个长度就是该字符串的属性。

5.6 对象的方法(function)

对象所具有的行为也可以称作方法。
比如，对字符串 **'a-b-c-d'** 进行拆分，这个 **拆分**就可以说是方法。

在 Python 中，方法的本质是函数，在类中定义的函数叫作方法，没在类中定义的函数就叫作函数。

在后面的章节中将讲解一些内建的函数或方法，为方便读者阅读，统一都叫作函数。

5.7 对象与变量

在编程过程中，很多时候需要给对象设置变量，相当于给对象贴一个标签，这样更方便识别。

比如 a = 1，表示给对象 1 贴一个标签，在引用变量 a 进行代码编写时，就相当于在使用对象 1。

在命名变量时，需要注意如下规则：

- 变量名可以由字母、数字、下画线 `_` 组成，但不能以数字开头；
- 变量名不能是 Python 关键字，但可以包含关键字；
- 变量名不能包含空格；
- 变量名尽量取得有意义，容易让人识别。

5.8 Python 中的数字与字符串

Python 中万物皆对象，数字与字符串只是其中的两种对象。

为什么要先学这两种对象呢？因为这两种对象最常用，也比较容易学习。在后面的章节中将陆续讲解更多的对象。

5.8.1 数字

Python 中的数字有 3 种类型：**整数、浮点数(小数）、复数**。

有时需要对数字进行转换，可以使用对象的函数，转换为整数使用 `int` 函数，转换为小数使用 `float` 函数，转换为复数使用 `complex` 函数。

将字符串 '99' 赋值给 `num` 变量，看看不同函数对 `num` 变量处理的不同结果。

```
num = '99'
print(num)  # 返回 '99'
print(type(num)) # 返回<class  'str' >

print(int(num)) # 返回 99
print(type(int(num))) # 返回<class  'int' >

print(float(num)) # 返回 99.0
print(type(float(num))) # 返回<class  'float' >

print(complex(num)) # 返回 (99+0j)
print(type(complex(num))) # 返回<class  'complex' >
```

5.8.2 字符串

字符串就是一串字符，是一个及以上字符的集合。

Python 中的字符串必须被一对单引号('')或双引号("")包围起来。要将其他数据转换为字符串类型，可以使用 `str()` 函数。

比如，将数字转换为字符串类型。

```
num = 99
print(str(num)) # 返回 99
print(type(str(num))) # 返回<class  'str' >
# 在 Python 中，还有一些常用的特殊字符，比如换行符（\n）、制表符（\t）、回车符（\r）等。在遇到特殊字符，需要将其转换为普通字符时，在其前面加上"\"即可。
# 还有另一种转换方法是，在字符串的左外侧加上字母 r（大小写均可）。
print('我是谁! \n 我在哪儿! ') # \n 表示换行
print('我是谁! \\n 我在哪儿! ') # \\n 被识别为普通字符
print(r'我是谁! \n 我在哪儿! ') # \n 被识别为普通字符
```

5.9 算术运算符

Python 中的常用算术运算符有加(+)、减(-)、乘(*)、除(/)、取模(%)、幂(**)、取整数(//)。下面具体介绍这些运算符。

5.9.1 加(+)

加运算符是执行数字相加运算的符号，比如运行代码 `print(100+199)`，将返回数字 299。

同时，加运算符也可以进行字符串的连接运算，比如运行代码 `print('杨辉'+'99 分')`，返回文本 **杨辉 99 分**。

```
print(100 + 199) # 返回数字 299
print('杨辉' + '99 分') # 返回文本 杨辉 99 分
```

除此之外，加号还可以在 **列表、元组** 等对象中做连接。

5.9.2 减(-)

减运算符是执行数字相减运算的符号，比如运行代码 `print(100-99)`，返回结果 1 。

```
print(100-99) # 返回结果 1
print('100' - 99) # 不能正确计算
```

如果相减的两个值中有一个值不是标准数字，就不能正确进行计算。

运行代码 `print('100' - 99)` 后，提示"unsupported operand type(s) for - : 'str' and 'int' "。

意思是一个值为字符串型，另一个值为整型，这两种数据类型不能在一起运算。

5.9.3 乘(*)

乘运算符是执行数字相乘运算的符号，比如运行代码 `print(100*99)`，返回 9900。乘运算符也有**重复** 的作用，可对字符串重复进行运算，比如运行代码 `print('python! '*3)`， 返回"python! python! python! "。

```
print(100*99) # 返回 9900
print('python! '*3) # 返回 python! python! python!
```

除此之外，乘号还可以用于重复其他对象，比如 **列表、元组** 等。

5.9.4 除(/)

除运算符是执行数字相除运算的符号，比如运行代码 `print(63/8)`，返回 7.875。

```
print(63/8) # 返回数字 7.875
```

> 注意，相除的结果为 float 型，即使商是整数，但其类型也是浮点型(小数) 。

5.9.5 取模(%)

取模运算符是执行数字相除运算后取余数的符号，比如运行代码 `print(63%8)`，返回 7，这个值便是 63 除以 8 的余数。

```
print(63%8) # 返回余数 7
```

5.9.6 幂(**)

幂运算符是执行乘方运算的符号。`n**m` 是指 m 个 n 相乘，也叫 n 的 m 次方。比如运行代码 `print(4**8)`，表示 4 的 8 次方，返回 65536。

```
print(4**8) # 返回值为 65536
```

5.9.7 取整数(//)

取整运算符是执行数字相除运算后取商的整数的符号，比如运行代码 `print(63//8)`，直接相除的商为 7.875，只取商的整数部分，所以返回 7。

```
print(63//8) # 返回值为 7
```

5.10 比较运算符

比较运算符通常用于比较两个数值或两个表达式的大小，比较结果返回一个逻辑值(`True` 或 `False`)，条件成立返回 `True`，条件不成立返回 `False`。

Python 中的比较运算符有 **等于(==)、不等于(!=)、大于(>)、小于(<)、大于或等于(>=)、小于或等于(<=)**。

下面以数字为比较对象介绍比较运算符的使用方法。

5.10.1 等于(==)

等于运算符用来比较两个数值是否相等，比如运行代码 `print(9 == 9)`，返回逻辑值 True，又比如运行代码 `print(9==8)`，返回 False。

```
print(9==9) # 返回逻样值 True
print(9==8) # 返回逻辑值 False
```

5.10.2 不等于(!=)

不等于运算符用来比较两个数值是否不相等，比如运行代码 `print(9!=8)`，返回逻辑值 True，又比如运行代码 `print(9!=9)`，返回逻辑值 False。

```
print(9!=8) # 返回逻辑值 True
print(9!=9) # 返回逻辑值 False
```

5.10.3 大于(>)

大于运算符用来判断其左边的数值是否大于右边的数值，比如运行代码 `print(9>8)`，返回逻辑值 True；又比如运行代码 `print(9 > 9)`，返回逻辑值 False。

```
print(9>8) # 返回逻辑值 True
print(9>9) # 返回逻辑值 False
```

5.10.4 小于(<)

小于运算符用来判断其左边的数值是否小于右边的数值，比如运行代码 `print(8<9)`，返回逻辑值 True；又比如运行代码 `print(9<9)`，返回逻辑值 False。

```
print(8<9) # 返回逻辑值 True
print(9<9) # 返回逻辑值 False
```

5.10.5 大于或等于(>=)

大于或等于运算符用来判断其左边的数值是否大于或等于右边的数值，比如运行代码 `print(8>=9)`，返回逻辑值 False；又比如运行代码 `print(9>=9)`，返回逻辑值 True。

```
print(8>=9) # 返回逻辑值 False
print(9>=9) # 返回逻辑值 True
```

5.10.6 小于或等于(<=)

小于或等于运算符用来判断其左边的数值是否小于或等于右边的数值，比如运行代码 `print(8<=9)`，返回逻辑值 True；又比如运行代码 `print(9<=8)`，返回逻辑值 False。

```
print(8<=9) # 返回逻辑值 True
print(9<=8) # 返回逻辑值 False
```

5.11 赋值运算符

赋值运算符 (=) 表示将等号右侧的对象赋给等号左侧的变量。等号左、右两侧的关系。

5.11.1 赋值运算

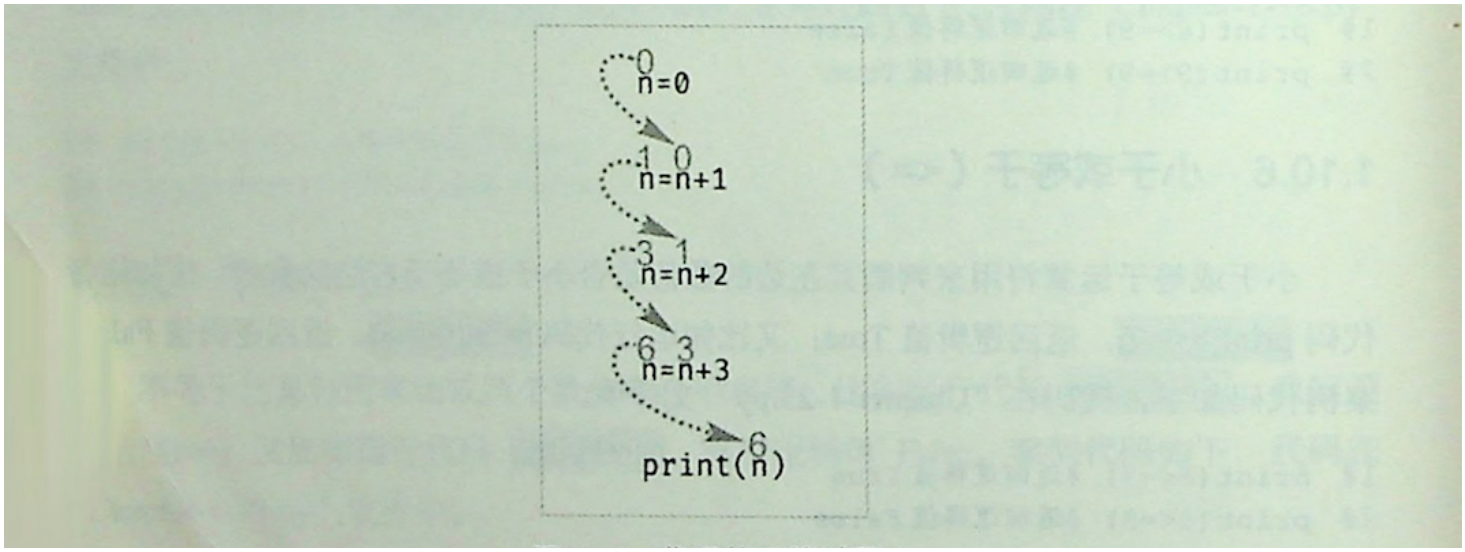
比如，`n=100` 表示变量 n 引用的对象是 100，`m=99` 表示变量 m 引用的对象是 99，代码 `print(n+m)` 表示将变量 n 引用的对象 100 与变量 m 引用的对象 99 相加，最后返回 199。

```
n = 100 # 将 100 赋值给变量 n
m = 99 # 将 99 赋值给变量 m
print(n+m) # 返回变量 n 与变量 m 加相的结果
```

5.11.2 累积式赋值运算

累积式赋值运算是编程中的一项重要技术。为了让读者更容易地理解累积式赋值运算的运算过程。

```
n = 0 # 变量 n 返回 0
n = n+1 # 变量 n 返回 1
n = n+2 # 变量 n 返回 3
n = n+3 # 变量 n 返回 6
print(n) # 在屏幕上输出变量 n 的值 6
```



除上面累积相加的累积式赋值运算外，还可以使用其他运算符做累积式赋值运算。

累积方式	写法 1	写法 2
加	n = n + 1	n+=1
减	n = n - 1	n-=1
乘	n = n * 100	n*=100
除	n = n / 100	n/=100
整除	n = n // 100	n//=100
取模	n = n % 100	n%=100
幂	n = n ** 100	n**=100

5.12 逻辑运算符

逻辑运算符一共有 3 个，分别是 **and、or 和 not**。

- and: 当 and 左右两边的条件都为真时，返回真(True）；否则，返回假(False）。
- or: 当 or 左右两边有一个条件为真时，返回真(True）；两个均为假，返回假(False）。
- not: 假的变成真的，真的变成假的，取反。

操作数 1	操作	操作数 2	结果
True	and	True	True
True	and	False	False
False	and	True	False
False	and	False	False

True	or	True	True
True	or	False	True
False	or	True	True
False	or	False	False
True	not		False
False	not		True

5.12.1 and（与）

当 and 运算符左右两边的条件都为真时，返回真(True）；当有一边的条件为假或两边的条件均为假时，返回假(False）。

```
print(True and True) # 左右两边均为 True，返回 True
print(100==100 and 10<11) # 对应案例
print(True and False) # 左边为 True，右边为 False，返回 False
print(100==100 and 10>11) # 对应案例
print(False and True) # 左边为 False，右边为 True，返回 False
print(100>100 and 10<11) # 对应案例
print(False and False) # 左右两边均为 False，返回 False
print(100>100 and 10>11) # 对应案例
```

5.12.2 or（或）

当 or 运算符左右两边任意一个条件为真时，返回真(True）；两个条件均为假，返回假(False）。

```
print(True or True) # 左右两边均为 True，则返回 True
print(100==100 or 10<11) # 对应案例
print(True or False) # 左边为 True，右边为 False，则返回 True
print(100==100 or 10>11) # 对应案例
print(False or True) # 左边为 False，右边为 True，则返回 True
print(100>100 or 10<11) # 对应案例
print(False or False) # 左右两边均为 False，则返回 False
print(100>100 or 10>11) # 对应案例
```

5.12.3 not（非）

如果对 True 取反，则返回 False；如果对 False 取反，则返回 True。

```
print(not True) # 如果对 True 取反，则返回 False
print(not 100==100) # 对应案例
print(not False) # 如果对 False 取反，则返回 True
```

5.13 成员运算符

除前面几个小节中讲解的算术运算符、比较运算符、赋值运算符、逻辑运算符外，Python 还支持使用成员运算符。

成员运算符用于测试字符串、列表等对象中是否包含指定的值。

成员运算符用 **in** 表示，返回值是逻辑值。

如果在指定的对象中找到了指定的值，则返回 True；否则返回 False。也可以使用 **not in** 来测试对象中没有指定的值。

```
print('杨辉' in  'IT 部杨辉 2000') # 返回 True
print('杨辉' not in  'IT 部杨辉 2000') # 返回 False
print('杨小辉' in  'IT 部杨辉 2000') # 返回 False
print('杨小辉'  not in  'IT 部杨辉 2000') # 返回 True
```

除可以在字符串中使用 in 运算符外，后面章节中将要学习的列表、集合、字典等对象也可以使用 in 运算符来做判断测试。

5.14 格式化字符串

在 Python 中，经常会对各种对象进行格式化处理。本节将使用 **format** 函数格式化指定的值，并将其插入字符串的占位符内。

5.14.1 使用位置和关键字格式化字符串

在使用 **format** 函数进行格式化时，使用花括号 **{}** 定义占位符，下面代码的返回值均为"恭喜杨辉获得 100 分。"

```
# 使用位置索引
# 按默认顺序获取 format 中的数据
print('恭喜 {} 获得 {} 分。' .format('杨辉' , 100))
# 按指定顺序获取 format 中的数据
print('恭喜 {0} 获得 {1} 分。' .format('杨辉' , 100))
# 使用关键字索引
# 按指定名称获取 format 中的数据
print('恭喜 {name} 获得 {score} 分。'.format(name='杨辉' , score=100))
```

也可以下面这么写：

```
name = '杨辉'
score = 100
print(f'恭喜 {name} 获得 {score} 分。')
```

```
name = '杨辉'
score = 100
print('恭喜 %s 获得 %d 分。' % (name, score))
```

5.14.2 数字格式设置

数字格式设置是常用设置，对数字格式化后返回的结果是字符串型数字。

```
print('{:.2f}'.format(3.1415926)) # 返回 3.14
print('{:.2%}'.format(0.1415926)) # 返回 14.16%
```

- : 表示要设置的值。
- .2 表示保留小数点后两位数。
- f 表示返回浮点数，也就是小数。
- % 表示设置成百分比格式。

5.14.3 对齐设置

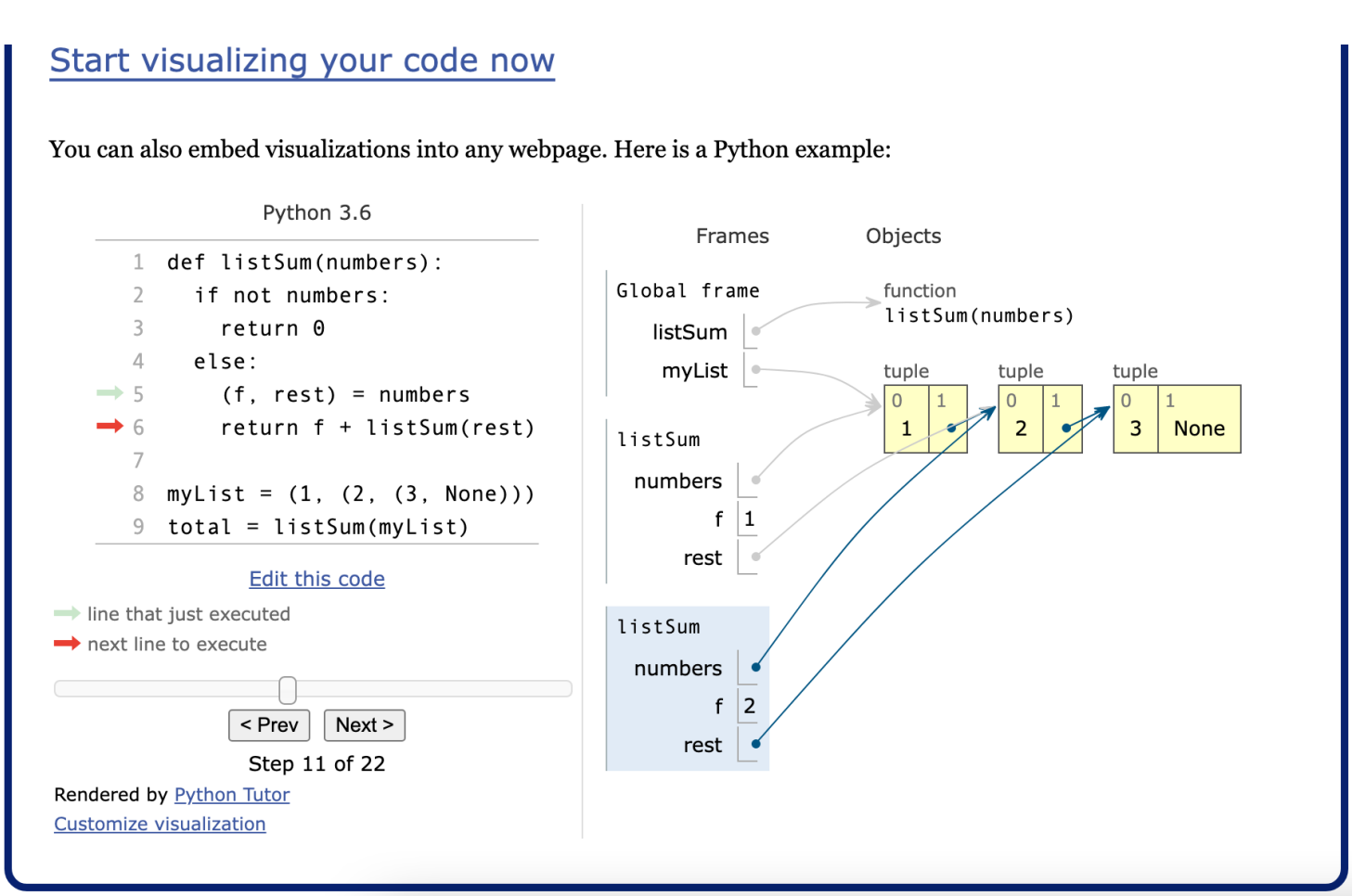
对齐设置是常用的格式化字符串的方式。

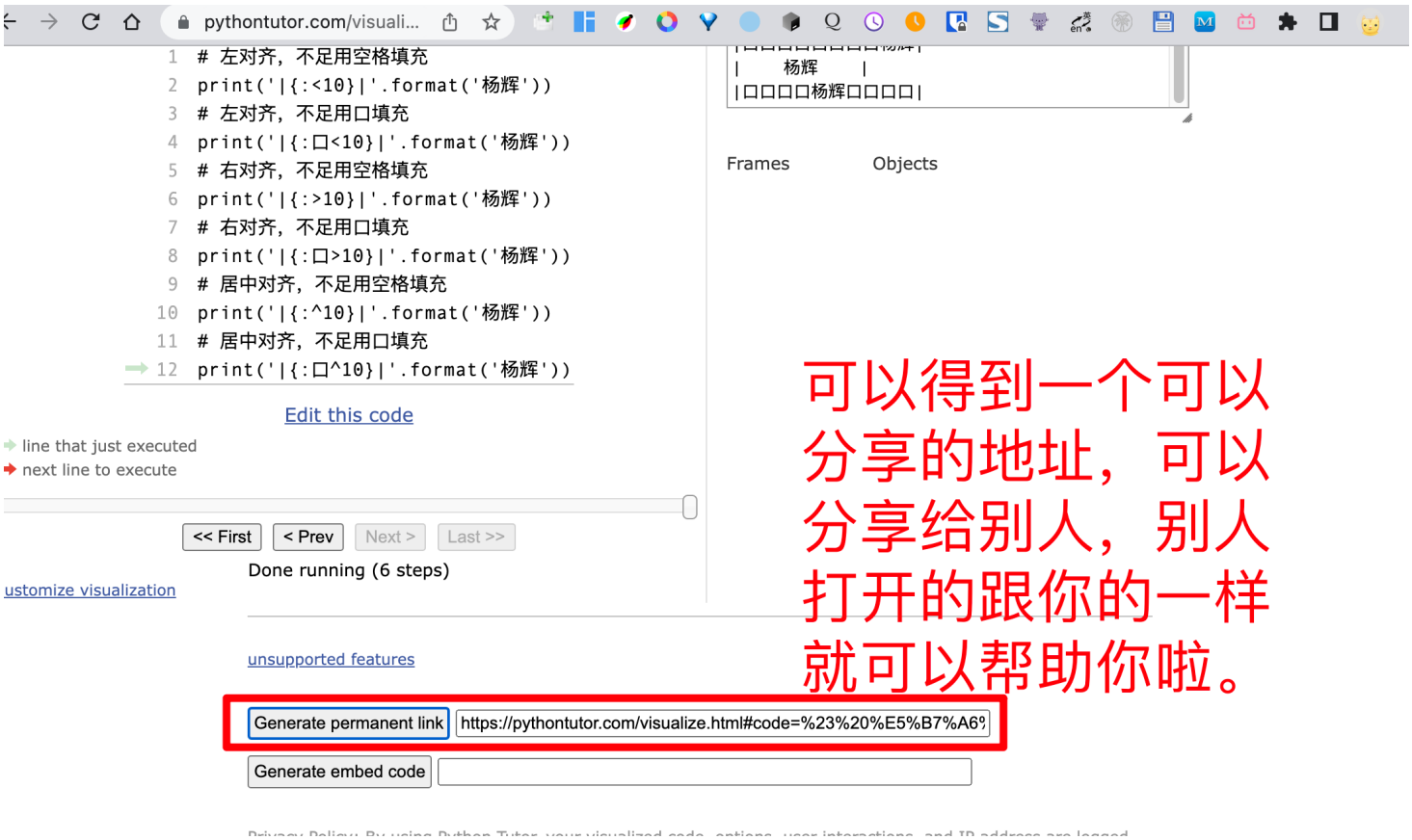
```
# 左对齐，不足用空格填充
print('|{:<10}|'.format('杨辉'))
# 左对齐，不足用口填充
print('|{: 口<10}|'.format('杨辉'))
# 右对齐，不足用空格填充
print('|{:>10}|'.format('杨辉'))
# 右对齐，不足用口填充
print('|{: 口>10}|'.format('杨辉'))
# 居中对齐，不足用空格填充
print('|{:^10}|'.format('杨辉'))
# 居中对齐，不足用口填充
print('|{: 口 ^10}|'.format('杨辉'))
```

- < 表示左对齐。
- > 表示右对齐。
- ^ 表示居中对齐。

5.15 pythontutor 使用

网址直达PythonTutor





6. Python 模块、包、库，三方库安装

6.1 什么是模块、包、库

- 模块(Module)
  - 模块是一个 Python 文件，扩展名为.py。
  - 在模块中能够组织 Python 代码段，把相关的代码放到一个模块里能让代码更好用、更易懂。
  - 在模块里能定义函数、类和变量，模块中也能包含可执行的代码。
- 包(Package)
  - 包是模块之上的概念，为了方便管理.py 模块文件，可以进行打包。
  - 包其实就是文件夹，只不过该文件夹下有名称为 `__init__.py` 的文件，否则就是普通的文件夹。
  - 包中可以有模块和子文件夹，假如子文件夹中也有 `__init__.py` 文件，那么它就是这个包的子包。
- 库(Library)
  - 在 Python 中，具有某些功能的模块和包都可以被称作库。

模块由诸多函数组成，包由诸多模块组成，库中可以包含模块、包和函数。

Python 中的库分为标准库和第三方库。

- 标准库就是安装 Python 时自带的库，可以直接使用。
- 第三方库是由第三方机构发布的，使用前需要安装。

6.2 Excel 读取库 xlrd

想用 Python **处理** Excel 数据，但是 Python 标准库中没有处理 Excel 文件的库，这时就需要安装第三方库。

> 如何使用 pip 在 Python 中进行库管理

`xlrd` 就是处理 Excel 文件的第三方库。下面介绍如何安装第三方库 `xlrd`。

> 不同的 IDE 和环境中有不同的安装方法，这个要根据环境的不同区别对待。

注意: 下面这条语句是为了演示一个错误的用法，你完全可以不运行下面的语句，而直接跳过下一句，到按照版本安装的方法。

```
pip install xlrd
```

```
import xlrd # 导入xlrd
wb = xlrd.open_workbook(r'./1-Scores.xlsx') # 读取工作簿对象。
```

```
---------------------------------------------------------------------------
XLRDError                                 Traceback (most recent call last)
<ipython-input-5-cd3c67ea92c8> in <module>
      1 import xlrd # 导入xlrd
----> 2 wb = xlrd.open_workbook(r'./1-Scores.xlsx') # 读取工作簿对象。

~/.local/lib/python3.6/site-packages/xlrd/__init__.py in open_workbook(filename, logfile, v
erbosity, use_mmap, file_contents, encoding_override, formatting_info, on_demand, ragged_ro
ws, ignore_workbook_corruption)
    168     # files that xlrd can parse don't start with the expected signature.
    169     if file_format and file_format != 'xls':
--> 170         raise XLRDError(FILE_FORMAT_DESCRIPTIONS[file_format]+'; not supported')
    171
    172     bk = open_workbook_xls(

XLRDError: Excel xlsx file; not supported
```

原因是最近 xlrd 更新到了 2.0.1 版本，只支持.xls 文件。

可以安装旧版 xlrd：

```
pip uninstall xlrd
```

```
pip install xlrd==1.2.0
```

6.2.1 xlrd 模块导入

安装好 **xlrd** 库后，要使用库中的功能，就需要导入 **xlrd**。

可以使用 **import** 语句来完成导入。导入的方法有以下两种。

- 方法 1: import 模块名1 [as 别名1] , 模块名2 [as 别名2]
  - 使用这种语法格式的 **import** 语句，可以导入库中的所有成员(变量、函数、类等）。
  - 如果模块名称太长，也可以用"**as 别名**"的方式来重命名。
  - 代码 `import xlrd` 将导入 **xlrd** 库的全部，这样就可以使用 **xlrd** 库中的所有功能了。
- 方法 2: from 模块名 import 成员名1 [as 别名1] , 成员名2 [as 别名2]
  - 使用这种语法格式的 **import** 语句，只导入库中指定的模块，而不是全部成员。
  - 代码 `from xlrd import open_workbook as owk` 则导入 xlrd 库中的 **open_workbook** 函数，别名为 **owk**，后面再次使用 **open_workbook** 函数时，可以使用 **owk** 来代替它。

```
import xlrd # 导入 xlrd
from xlrd import open_workbook as owk # 导入 xlrd 中的 open_workbook 函数
```

6.2.2 读取 Excel 工作表

Excel 的位置在不同的系统中有不同的表示：

- Mac 中：/Users/linken/Desktop/PythonExcelOperation
- Windows 中：C:\Desktop\
- 网络地址：https://github.com/WangLaoShi/Aliyun-Python/blob/main/Files/1-Scores.xlsx?raw=true

> **斜杠的位置不同**

```
import xlrd # 导入 xlrd
wb = xlrd.open_workbook(r'1-Scores.xlsx') # 读取工作簿对象。
```

上面的代码是读取加载 Excel 文件，除了这个还可以实现很多关于 Excel 文件的操作。

```
import xlrd # 导入 xlrd
xlrd.xlsx.ensure_elementtree_imported(False, None)
xlrd.xlsx.Element_has_iter = True
filePath = r'1-Scores.xlsx'

# 读取工作簿对象。
wb = xlrd.open_workbook(filePath)

# 读取工作簿下的所有工作表对象
all_ws1 = wb.sheets()

# 读取工作簿下的所有工作表对象的名称
all_ws2 = wb.sheet_names()

# 用索引值读取工作表 - 方法 1
ws1 = wb.sheet_by_index(0)

# 用索引值读取工作表 - 方法 2
ws2 = wb.sheets()[1]

# 用索引值读取工作表 - 方法 3
ws3 = wb.sheet_by_name('雪豹队')

# 直接通过工作簿读取工作表对象
ws4 = xlrd.open_workbook(filePath).sheet_by_name('飞龙队')
```

6.2.3 读取 Excel 行、列、单元格信息

```
import xlrd # 导入 xlrd

# 读取工作簿对象。
```

```
filePath = r'1-Scores.xlsx'
wb = xlrd.open_workbook(filePath)

# 读取工作表对象。
ws = wb.sheet_by_name('飞龙队')

# 返回工作表中已使用的行数。
row_count = ws.nrows

# 返回工作表中已使用的列数。
col_count = ws.ncols

# 返回工作表中指定行已使用的单元格对象。
row_obj = ws.row(1)

# 返回工作表中指定行已使用的单元格的值。
row_val = ws.row_values(1)

# 返回工作表中指定列已使用的单元格对象。
col_obj = ws.col(0)

# 返回工作表中指定列已使用的单元格的值。
col_val = ws.col_values(0)

# 返回工作表中指定行、列交叉单元格对象。
cell_obj = ws.cell(3,1)

# 返回工作表中指定行、列交叉单元格的值。
cell_val = ws.cell_value(3,1)
```

6.3 Excel 写入库 xlwt

前面介绍的 **xlrd** 库只能**读取** Excel 文件中的数据，如果要往 Excel 文件中**写入**数据，xlrd 库则**不**具备这个功能，需要安装 **xlwt** 库。

xlwt 库具有创建工作簿、工作表，以及将数据写入单元格的功能。

```
pip install xlwt
```

6.3.1 新建工作簿、新建工作表和写入数据

> Python 需要有写入文件权限，特别是 Linux 和 Mac 中

```
import xlwt # 导入 xlwt

# 新建工作簿
nwb = xlwt.Workbook('utf-8')

```

```
# 在新建工作簿中创建工作表
nws = nwb.add_sheet('工资表')

# 在工作表的指定单元格写入值
nws.write(0,0,'张三: 9000 元')

# 保存工作簿
nwb.save('工资表.xls')
```

6.4 Excel 修改库 xlutils

xlrd 库只能用于**读取**已经存在的 Excel 工作簿、工作表、单元格等相关信息，而 **xlwt** 库**只能新建**工作簿、新建工作表、将数据写入单元格等，没有办法对现有的工作簿进行**修改**。

要实现修改功能，就需要安装 **xlutils** 库，相当于在 **xlrd** 库和 **xlwt** 库之间建立起一座桥梁。但要注意，使用 xlutils 库一定要先安装 **xlrd** 库和 **xlwt** 库，否则安装 **xlutils** 库没有意义。

```
pip install xlutils
```

```
import xlrd # 导入 xlrd
from xlutils.copy import copy # 导入 xlutils 下 copy 模块下的 copy 函数

filePath = r'2-Salary.xls'

# 读取工作簿
wb = xlrd.open_workbook(filePath)

# 复制工作簿
nwb = copy(wb)

# 用索引号读取工作簿中的工作表
ws1 = nwb.get_sheet(0)

# 用名称读取工作簿中的工作表
ws2 = nwb.get_sheet('工资表')

# 在工作簿中新建工作表
ws3 = nwb.add_sheet('汇总表')

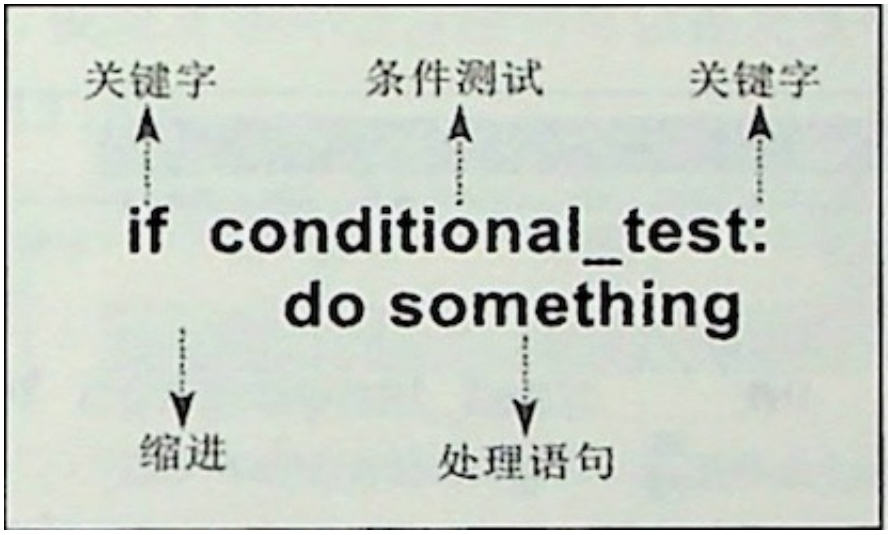
# 将数据写入单元格
ws3.write(0,0,'总计')

# 将数据写入单元格
ws3.write(0,1,12000)

# 保存工作簿
nwb.save('2-Salary-2.xls')
```

7. 条件语句

7.1 if 语句



- `if` 是关键字，固定写法，不能有任何变化。
- `conditional test` 是条件测试表达式，返回的结果是逻辑值(True 或 False）。
- `:` 是关键字，固定写法，不能有任何变化。
- `do something` 是条件测试表达式返回的值为 True 时执行的处理语句。

```
if expression:
    expr_true_suite
```

- if 语句的 `expr_true_suite` 代码块只有当条件表达式 `expression` 结果为真时才执行，否则将继续执行紧跟在该代码块后面的语句。
- 单个 if 语句中的 `expression` 条件表达式可以通过布尔操作符 `and`， `or` 和 `not` 实现多重条件判断。

7.1.1【例子】if 标准用法

```
if 2 > 1 and not 2 > 3:
    print('Correct Judgement!')

# Correct Judgement!
```

7.1.2【例子】if 标准用法

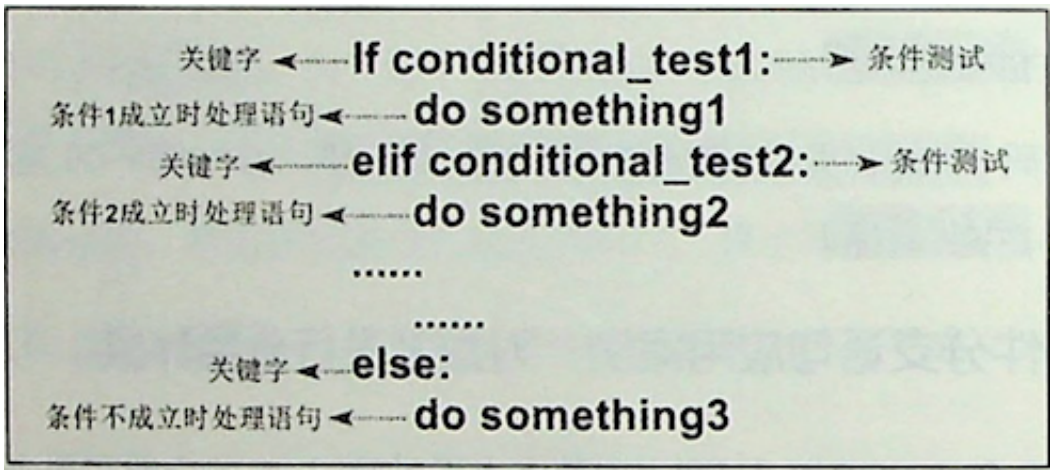
```
if 100>=90: # 条件成立
    print('优秀') # 执行处理语句
if 80>=90: # 条件不成立
    print('优秀') # 不执行处理语句
```

7.1.3【综合案例】根据分数判断等级

```
import xlrd # 导入 xlrd 库。
wb = xlrd.open_workbook('./3-Scores.xlsx') # 读取工作簿。
ws = wb.sheet_by_name('成绩表') # 读取工作表。
col_vals = ws.col_values(1) # 获取工作表指定列已用单元格区域的值。
for score in col_vals: # 循环列区域的每个值。
    if type(score)==float and score>=90: # 判断条件表达式是否成立。
        print(score,'优秀') # 条件成立，则执行。
```

- 第 1~4 行代码为数据的读取做准备工作，将工作表中分数列的所有数据读取到 **col_vals** 变量中。
- 第 5 行代码 `for score in col_vals`:，将 col_vals 变量中的每个分数循环读取给 score 变量。
- 第 6 行代码 `type(score) == float and score>=90:`，在 for 循环体中，首先判断 score 变量中的分数是否是 float 类型。为什么要做类型判断？B 列的第 1 个值"分数"是汉字，并不是数字，因此要做类型判断，是数字类型并且分数大于或等于 90，才能执行 `print(score,'优秀')` 语句。
- 第 7 行代码 `print(score, "优秀")`，如果第 6 行的条件成立，则执行该行代码。

7.2 if - else 分支语句




```
if expression:
    expr_true_suite
else:
    expr_false_suite
```

- Python 提供与 if 搭配使用的 else，如果 if 语句的条件表达式结果布尔值为假，那么程序将执行 else 语句后的代码。

7.2.1【例子】猜一猜小姐姐想的是哪个数字?

```
temp = input("猜一猜小姐姐想的是哪个数字？ ")
guess = int(temp) # input 函数将接收的任何数据类型都默认为 str。
if guess == 666:
    print("你太了解小姐姐的心思了！ ")
    print("哼，猜对也没有奖励！ ")
else:
    print("猜错了，小姐姐现在心里想的是 666！ ")
print("游戏结束，不玩儿啦！ ")
```

`if` 语句支持嵌套，即在一个 `if` 语句中嵌入另一个 `if` 语句，从而构成不同层次的选择结构。

7.2.2【例子】`else` 的缩进问题

Python 使用缩进而不是大括号来标记代码块边界，因此要特别注意 `else` 的缩进问题。

```
hi = 6
if hi > 2:
    if hi > 7:
        print('好棒！ 好棒！')
else:
    print('切 ~')

# 无输出
```

7.2.3【例子】猜一猜小姐姐想的是哪个数字？

```
temp = input("猜一猜小姐姐想的是哪个数字？ ")
guess = int(temp)
if guess > 8:
    print("大了，大了")
else:
    if guess == 8:
        print("你太了解小姐姐的心思了！ ")
        print("哼，猜对也没有奖励！ ")
    else:
        print("小了，小了")
print("游戏结束，不玩儿啦！ ")
```

7.3 if - elif - else 分支语句

```
if expression1:
    expr1_true_suite
elif expression2:
    expr2_true_suite
    .
    .
elif expressionN:
```

```
    exprN_true_suite
else:
    expr_false_suite
```

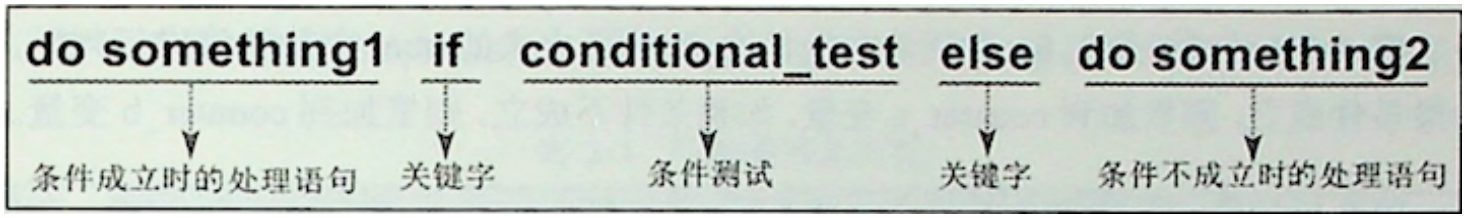
- elif 语句即为 else if，用来检查多个表达式是否为真，并在为真时执行特定代码块中的代码。

【例子】

```
temp = input('请输入成绩:')
source = int(temp)
if 100 >= source >= 90:
    print('A')
elif 90 > source >= 80:
    print('B')
elif 80 > source >= 60:
    print('C')
elif 60 > source >= 0:
    print('D')
else:
    print('输入错误! ')
```

7.4 if 条件分支语句单行写法

实际上，if···else···条件分支语句可以写在一行，也叫**三目运算**。如果判断后的处理语句不太复杂，则可以使用这种写法。




```
print('优秀') if 100>=90 else print('普通') #当前条件成立，在屏幕上打印 " 优 秀"
print('优秀') if 80>=90 else print(' 普通 ') #当前条件不成立，在屏幕上打印 " 普 通 "
```

- 第 1 行代码 print('优秀') if 100>=90 else print(' 普通 ')，因为 100>=90 是成立的，所以会执行代码 print(' 优秀')。
- 第 2 行代码 print('优秀') if 80>=90 else print(' 普通 ')，因为 80>=90 是不成立的，所以会执行代码 print(' 普通')。

7.5 assert 关键词

- `assert` 这个关键词我们称之为"断言"，当这个关键词后边的条件为 **False** 时，程序自动崩溃并抛出 `AssertionError` 的异常。

【例子】

```
my_list = ['lsgogroup']
my_list.pop(0)
assert len(my_list) > 0

# AssertionError
```

【例子】在进行单元测试时，可以用来在程序中置入检查点，只有条件为 True 才能让程序正常工作。

```
assert 3 > 7

# AssertionError
```

7.6 案例：对数字进行分类计数

统计［95，89，69，100，88，94，91］列表中数字大于或等于 90 的个数和数字小于 90 的个数。

```
lst=[95,89,69,100,88,94,91] # 要被循环判断的数字列表。
counter_a,counter_b=0,0 # 分别初始化 counter_a 和 counter_b 变量值为 0。
for num in lst: # 循环 lst 变量中的数字。
    if num>=90: # 如果 num 大于等于 90。
        counter_a +=1 # 条件成立则将计数到 counter_a 变量中。
    else: # 如果 num 不大于等于 90。
        counter_b +=1# 条件不成立则将计数到 counter_b 变量中。
print('>=90 有 {} 个，<90 有 {} 个。'.format(counter_a,counter_b))# 统计>=90 与<90 的数字个数。
```

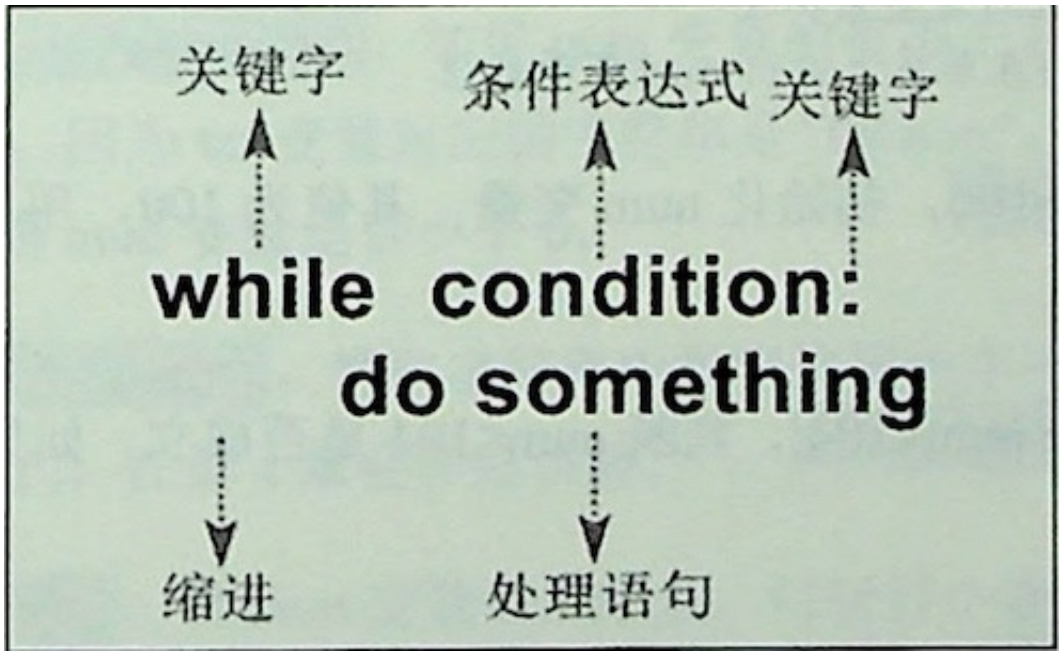
- 第 1 行代码 lst＝［95,89,69,100,88,94,91］，将列表赋值给 lst 变量。
- 第 2 行代码 counter_a，counter_b＝0,0，分别初始化 counter_a 和 counter_b，用于在后面做判断时存储条件成立的次数和条件不成立的次数。
- 第 3 行代码 for num in lst，循环读取 lst 列表中的数字，赋值给 num 变量。
- 第 4～7 行代码，是在 for 循环体中执行的，对循环出来的 num 变量的值进行判断，如果条件成立，则累加到 counter_a 变量，如果条件不成立，则累加到 counter_b 变量。
- 第 8 行代码 print('>=90 有{}个，<90 有 {} 个。'.format(counter_a，counter_b))，在屏幕上输出统计结果，最后返回的结果为">=90 有 4 个，＜90 有 3 个。"

8. 循环语句

8.1 while 循环

`while` 语句最基本的形式包括一个位于顶部的布尔表达式，一个或多个属于 `while` 代码块的缩进

语句。



- `while` 是关键字，固定写法，不能有任何变化。
- `condition` 是条件表达式，需要返回一个逻辑值 True 或 False。
- `:` 是关键字，可以理解为到此为止，后面不能再加任何代码，也是换行的标志。
- `do something` 是 while 循环的处理语句，具体如何处理、代码如何编写要根据特定的需求而定。这些处理语句都必须做缩进处理，不能与 while 关键字对齐。

```
while 布尔表达式:
    代码块
```

- `while` 循环的代码块会一直循环执行，直到布尔表达式的值为布尔假。
- 如果布尔表达式不带有 `<、>、==、！=、in、not in` 等运算符，仅仅给出数值之类的条件，也是可以的。
- 当 `while` 后写入一个非零整数时，视为真值，执行循环体；写入 `0` 时，视为假值，不执行循环体。
- 也可以写入 `str、list` 或任何序列，长度非零则视为真值，执行循环体；否则视为假值，不执行循环体。

```
num = 100 # 初始化 num 变量的值为 100
while num<104: # 当 num 小于 104 时循环
    num +=1 # 每次对变量累加 1
    print(num) # 在屏幕打印 num 变量的值
```

- 第 1 行代码 `num =100`，初始化 num 变量，其值为 100，用于在 while 循环体中做累加赋值运算。

- 第 2 行代码 `while num<104:`，判断 num<104 是否成立，如果条件成立，则执行循环体中的语句。
- 第 3 行代码 `num +=1`，对 num 变量每次加 1，然后赋值给 num 变量。由于此行代码是在循环体内的，所以 while 循环体循环多少次就累加赋值多少次。
- 第 4 行代码 `print(num)`，在屏幕上输出 num 变量的值，该行代码是在循环体内的，所以打印输出多少次取决于该循环体循环多少次。

8.1.1 【例子】猜一猜小姐姐想的是哪个数字?

```
count = 0
while count < 3:
    temp = input("猜一猜小姐姐想的是哪个数字? ")
    guess = int(temp)
    if guess > 8:
        print("大了, 大了")
    else:
        if guess == 8:
            print("你太了解小姐姐的心思了! ")
            print("哼, 猜对也没有奖励! ")
            count = 3
        else:
            print("小了, 小了")
    count = count + 1
print("游戏结束, 不玩儿啦! ")
```

8.1.2【例子】循环字符串

```
txt =  'Python' # 要被循环的字符串。
num = 0 # 初始化 num 变量为 0。
while num<len(txt): # num 变量小于 txt 的字符长度时则开始循环。
    print(txt[num]) # 打印提取的每个字符。
    num += 1 # 对 num 变量进行累加。
```

8.1.3【例子】布尔表达式返回 0，循环终止

```
string = 'abcd'
while string:
    print(string)
    string = string[1:]

# abcd
# bcd
# cd
# d
```

8.2 while - else 循环

```
while 布尔表达式:
    代码块
else:
    代码块
```

当 `while` 循环正常执行完的情况下，执行 `else` 输出，如果 `while` 循环中执行了跳出循环的语句，比如 `break`，将不执行 `else` 代码块的内容。

8.2.1【例子】比 5 大，还是比 5 小

```
count = 0
while count < 5:
    print("%d is  less than 5" % count)
    count = count + 1
else:
    print("%d is not less than 5" % count)

# 0 is  less than 5
# 1 is  less than 5
# 2 is  less than 5
# 3 is  less than 5
# 4 is  less than 5
# 5 is not less than 5
```

8.2.2【例子】增加一个 break 会怎么样

```
count = 0
while count < 5:
    print("%d is  less than 5" % count)
    count = 6
    break
else:
    print("%d is not less than 5" % count)

# 0 is  less than 5
```

8.3 【综合训练】使用 while 循环批量创建 Excel

```
import xlwt # 导入 xlwt 库。
wb = xlwt.Workbook('utf-8') # 新建工作簿。
year_num = 2010 # 初始化 year_num 变量为 2010。
while year_num < 2020: # 如果 year_num 变量小于 2020 则执行 while 循环体的语句。
    txt = '{}年业绩表'.format(year_num) # 使用变量 year_num 格式化为工作表名称。
    wb.add_sheet(txt) # 新建工作表。
```

```
7       year_num += 1 # 累加 year_num 变量。
8   wb.save('./PerformanceWorksheet.xls') # 保存工作簿。
```

- 第 1 行代码 import xlwt，导入需要用到的 xlwt 库。
- 第 2 行代码 wb = xlwt. Workbook('utf-8')，新建一个工作簿并赋值给 wb 变量，此工作簿在循环新建工作表时使用。
- 第 3 行代码 year_num=2010，初始化 year_num 变量，其值为 2010，可以将此值看作年份。
- 第 4 行代码 while year_num<2020:，如果 year_num 变量的值小于 2020，则执行 while 循环体的语句。year_num 变量从 2010 累加到 2019，条件都成立。
- 第 5~7 行代码都属于 while 循环体中的语句。
  - 第 5 行代码 以 txt = '{}年业绩表'.format(year_num)，是将 year_num 变量格式化为工作表名称。
  - 第 6 行代码 wb.add_sheet(txt)，用来新建工作表，并将 txt 变量的值作为新建的工作表名称。
  - 第 7 行代码 year_num +=1，对 year_num 变量进行累加，每次加 1。
- 第 8 行代码 wb.save('./PerformanceWorksheet.xls')，当 while 循环结束后，执行工作簿的保存操作。注意，不要将此行代码放在 while 循环体中，否则循环一次就保存一次。

8.4 for 循环

`for` 循环是迭代循环，在 Python 中相当于一个通用的序列迭代器，可以遍历任何有序序列，如 `str、list、tuple` 等，也可以遍历任何可迭代对象，如 `dict` 。



- `for` 是关键字，固定写法，不能有任何变化。
- `item` 是元素，相当于变量名称，用户可自行命名，用于接收从 **iterable** 中循环出来的元素。
- `in` 是关键字，固定写法，不能有任何变化。
- `iterable` 是迭代器，类似于一个容器，表达形式由用户自行定义，从该迭代器中读取出的数据

将传给 item 变量。

- `:` 也是关键字，可以理解为到此为止，后面不能再加任何代码，也是换行的标志。
- `do something` 是 for 循环的处理语句，具体如何处理、代码如何编写等要根据特定的需求而定。处理语句必须做缩进处理，不能与上面的 for 语句对齐。

```
for 迭代变量 in 可迭代对象:
    代码块
```

每次循环，迭代变量被设置为可迭代对象的当前元素，提供给代码块使用。

8.4.1 【例子】循环字符串

```
for i in 'ILoveLSGO':
    print(i, end='')  # end ='' 表示不换行输出

# I L o v e L S G O
```

8.4.2【例子】循环列表（直接法和通过长度）

```
member = ['林肯', '杨小辉', '谷爱凌', '苏翊鸣', '王菲']
for each in member:
    print(each)

for i in range(len(member)):
    print(member[i])
```

8.4.3【例子】循环访问字典（使用 items）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
for key, value in dic.items():
    print(key, value, sep=':', end=' ')
# a:1 b:2 c:3 d:4
```

8.4.4【例子】循环访问字典（使用 keys）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

for key in dic.keys():
    print(key, end=' ')

# a b c d
```

8.4.5【例子】循环访问字典（使用 values）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

for value in dic.values():
    print(value, end=' ')

# 1 2 3 4
```

8.4.6【例子】循环序列数（range）

循环序列数是常见的一种循环方式，下面介绍循环指定范围内的序列数。

要指定序列数的范围，可以使用 Python 中的 range 函数。

- 函数语法：
  - `range(start,stop,step)`
- 参数说明：
  - start: 表示起始值，默认从 0 开始。例如，range(5）等价于 range(0,5）。
  - stop: 表示终止值，但不包括终止值。例如，range(0,5）的返回值是 0、1、2、3、4，不包括 5。
  - step: 步长，默认为 1。例如，range(0,5）等价于 range(0,5,1），返回值是 0、1、2、3、4。如果是 range(0,5,2），则返回值是 0、2、4。

8.4.7【例子】循环批量创建 Excel 文件

```
import xlwt # 导入 xlwt
for month_num in range(1,13): # 循环 1 到 13 之间的数字
    month_name = '{}月份.xls'.format(month_num) # 格式化数字为工作簿名
    nwb = xlwt.Workbook('utf-8') # 新建工作簿
    nwb.add_sheet(month_name) # 在工作簿中新建工作表
    nwb.save(month_name) # 保存工作簿
```



8.5 for 循环嵌套语句

for 循环语句在某些时候需要进行多层嵌套。下面以打印乘法表为例，来讲解 **for** 循环语句的嵌套用法。

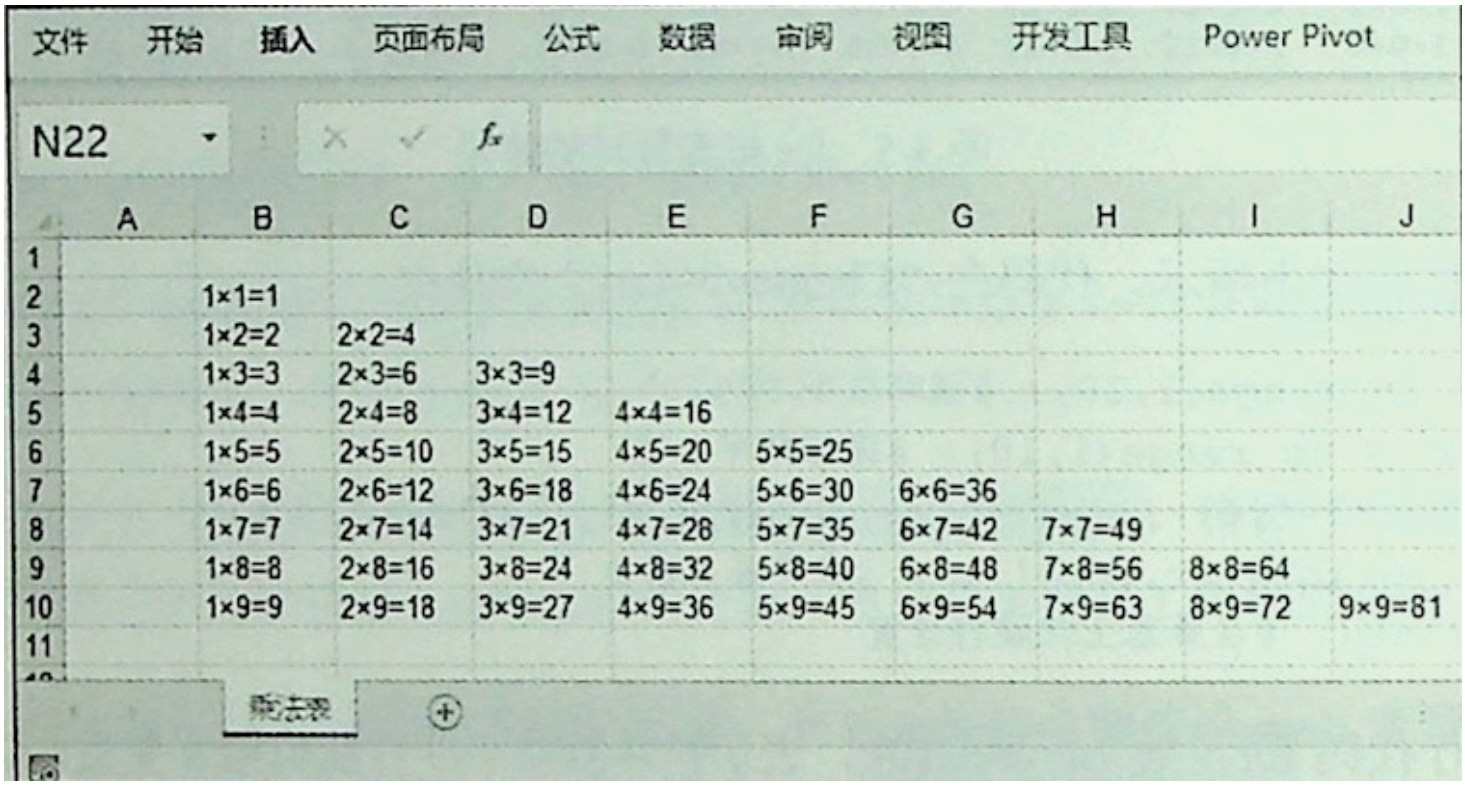
```
1 for x in range(1,10): # 循环数字 1~9
2     for y in range(1,10): # 循环数字 1~9
3         txt='{0}×{1}={2}'.format(y,x,x*y) # 格式化乘法表
4         print(txt,end='\t') # 在屏幕打印
5     print() # 在屏幕做换行设置
```

```
1×1=1   2×1=2   3×1=3   4×1=4   5×1=5   6×1=6   7×1=7   8×1=8   9×1=9
1×2=2   2×2=4   3×2=6   4×2=8   5×2=10  6×2=12  7×2=14  8×2=16  9×2=18
1×3=3   2×3=6   3×3=9   4×3=12  5×3=15  6×3=18  7×3=21  8×3=24  9×3=27
1×4=4   2×4=8   3×4=12  4×4=16  5×4=20  6×4=24  7×4=28  8×4=32  9×4=36
1×5=5   2×5=10  3×5=15  4×5=20  5×5=25  6×5=30  7×5=35  8×5=40  9×5=45
1×6=6   2×6=12  3×6=18  4×6=24  5×6=30  6×6=36  7×6=42  8×6=48  9×6=54
1×7=7   2×7=14  3×7=21  4×7=28  5×7=35  6×7=42  7×7=49  8×7=56  9×7=63
1×8=8   2×8=16  3×8=24  4×8=32  5×8=40  6×8=48  7×8=56  8×8=64  9×8=72
1×9=9   2×9=18  3×9=27  4×9=36  5×9=45  6×9=54  7×9=63  8×9=72  9×9=81
```

- 第 1 行代码 for x in range(1,10) ；，表示循环数字 1~9 并赋值给x变量，这是外层 for 循环。

- 第 2 行代码 for y in range(1,10），表示循环数字 1~9 并赋值给 y 变量，这是内层 for 循环。注意此循环是嵌套在外层循环中的，也就是说，外层 for 循环执行 1 次，内层 for 循环要执行 9 次。外层 for 循环总共需要执行 9 次，内层 for 循环要执行 81 次。属于内层 for 循环的第 3 行代码和第 4 行代码要执行 81 次。
- 第 3 行代码 txt = ('{0}×{1}={2}'.format(y,x,x*y)，使用 format 函数格式化两个乘数 x 和 y，以及乘积结果 x＊y，然后赋值给 txt 变量。
- 第 4 行代码 print(uxt,end=t），使用 print 函数将 tt 变量中的值输出到屏幕，end='\t' 表示以制表符结束，如果不特别指定，则默认以回车符结束。
- 第 5 行代码 print()，此行代码是与内层 for 循环语句同级的，属于外层 for 循环下的语句，它会执行 9 次，而不是执行 81 次。这行代码在每次内层 for 循环完成后，加一个回车符以起到换行的作用。前面说过 print 函数默认以回车符结束，所以没在 print 函数中做任何改动。

8.6 【综合训练】for 使用嵌套循环输出九九乘法表并保存在 Excel 中




```
import xlwt # 导入 xlwt 库
nwb=xlwt.Workbook('utf-8') # 新建工作簿
nws=nwb.add_sheet('乘法表') # 新建工作表
for x in range(1,10): # 循环数字 1 到 9
    for y in range(1,x+1): # 循环数字 1 到 x+1
        txt='{0}×{1}={2}'.format(y,x,x*y) # 格式化乘法公式
        nws.write(x,y,txt) # 将乘法公式写入单元格
nwb.save('./99Mul.xlsx') # 保存工作簿

```

- 第 1 行代码 import xlwt，导入需要用到的 xlwt 库。
- 第 2 行代码和第 3 行代码 nwb-xlwt. Workbook(utf-8）和 nws-nwb.add_ sheet(乘法表），

分别新建工作簿和在工作簿中新建工作表。

- 第 4 行代码 for x in range(1，10）:，这是外层 for 循环，循环数字 1~9，然后将循环出来的数字赋值给x变量。
- 第 5 行代码 for y in range(1，x+1）:，这是内层 for 循环，其中 x+1 表示终止循环的数字。如果外层 for 循环执行 1 次，则内层 for 循环为 range(1，1+1 次，即只能循环 1 次：如果外层 for 循环执行 2 次，则内层 for 循环为 range(1，2+1），即内层循环只能循环 2 次，以此类推。
- 第 6 行代码 txt ='{0} X {1}= {2}'.format(y，x，x＊y), 格式化循环出来的数字，组成乘法公式并赋值给 txt 变量。
- 第 7 行代码 nws.write(x，y，txt），将第 6 行代码中 txt 变量的值写入工作表中对应的单元格。
- 第 8 行代码 nwb.save('./99Mul.xlsx')，完成乘法表的写入后进行保存。注意，此行代码与外层 for 循环语句是同级的，因此该行代码只执行 1 次。如果将其放在外层 for 循环体中，就会执行 9 次：如果将其放在内层 for 循环体中，就会执行 45 次。虽然不影响结果，但更耗资源。

8.7 for - else 循环

```
for 迭代变量 in 可迭代对象:
    代码块
else:
    代码块
```

当 `for` 循环正常执行完的情况下，执行 `else` 输出，如果 `for` 循环中执行了跳出循环的语句，比如 `break`，将不执行 `else` 代码块的内容，与 `while – else` 语句一样。

【例子】

```
for num in range(10, 20):  # 迭代 10 到 20 之间的数字
    for i in range(2, num):  # 根据因子迭代
        if num % i == 0:  # 确定第一个因子
            j = num / i  # 计算第二个因子
            print('%d 等于 %d * %d' % (num, i, j))
            break  # 跳出当前循环
    else:  # 循环的 else 部分
        print(num, '是一个质数')

# 10 等于 2 * 5
# 11 是一个质数
# 12 等于 2 * 6
# 13 是一个质数
# 14 等于 2 * 7
# 15 等于 3 * 5
# 16 等于 2 * 8
# 17 是一个质数
# 18 等于 2 * 9
```

```
# 19 是一个质数
```

8.8 enumerate()函数

```
enumerate(sequence, [start=0])
```

- `sequence`：一个序列、迭代器或其他支持迭代对象。
- `start`：下标起始位置。
- 返回 enumerate(枚举) 对象

【例子】

```
seasons = ['Spring', 'Summer', 'Fall', 'Winter']
lst = list(enumerate(seasons))
print(lst)
# [(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
lst = list(enumerate(seasons, start=1))  # 下标从 1 开始
print(lst)
# [(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

`enumerate()` 与 for 循环的结合使用。

```
for i, a in enumerate(A)
    do something with a
```

用 `enumerate(A)` 不仅返回了 `A` 中的元素，还顺便给该元素一个索引值 (默认从 0 开始)。此外，用 `enumerate(A, j)` 还可以确定索引起始值为 `j` 。

【例子】

```
languages = ['Python', 'R', 'Matlab', 'C++']
for language in languages:
    print('I love', language)
print('Done!')
# I love Python
# I love R
# I love Matlab
# I love C++
# Done!


for i, language in enumerate(languages, 2):
    print(i, 'I love', language)
print('Done!')
# 2 I love Python
```

```
# 3 I love R
# 4 I love Matlab
# 5 I love C++
# Done!
```

8.9 break 语句

`break` 语句可以跳出当前所在层的循环。

【例子】

```
import random
secret = random.randint(1, 10) #[1,10]之间的随机数

while True:
    temp = input("猜一猜小姐姐想的是哪个数字? ")
    guess = int(temp)
    if guess > secret:
        print("大了, 大了")
    else:
        if guess == secret:
            print("你太了解小姐姐的心思了! ")
            print("哼, 猜对也没有奖励! ")
            break
        else:
            print("小了, 小了")
print("游戏结束, 不玩儿啦! ")
```

8.10 continue 语句

`continue` 终止本轮循环并开始下一轮循环。

【例子】

```
for i in range(10):
    if i % 2 != 0:
        print(i)
        continue
    i += 2
    print(i)

# 2
# 1
# 4
# 3
```

```
# 6
# 5
# 8
# 7
# 10
# 9
```

---

9. pass 语句

`pass` 语句的意思是"不做任何事"，如果你在需要有语句的地方不写任何语句，那么解释器会提示出错，而 `pass` 语句就是用来解决这些问题的。

【例子】

```
def a_func():

# SyntaxError: unexpected EOF while parsing
```

【例子】

```
def a_func():
    pass
```

`pass` 是空语句，不做任何操作，只起到占位的作用，其作用是为了保持程序结构的完整性。尽管 `pass` 语句不做任何操作，但如果暂时不确定要在一个位置放上什么样的代码，可以先放置一个 `pass` 语句，让代码可以正常运行。

---

10. 推导式

10.1 列表推导式

```
[expr for value in collection [if condition] ]
```

【例子】

```
x = [-4, -2, 0, 2, 4]
y = [a * 2 for a in x]
print(y)
# [-8, -4, 0, 4, 8]
```

【例子】

```
x = [i ** 2 for i in range(1, 10)]
print(x)
# [1, 4, 9, 16, 25, 36, 49, 64, 81]
```

【例子】

```
x = [(i, i ** 2) for i in range(6)]
print(x)

# [(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

【例子】

```
x = [i for i in range(100) if (i % 2) != 0 and (i % 3) == 0]
print(x)

# [3, 9, 15, 21, 27, 33, 39, 45, 51, 57, 63, 69, 75, 81, 87, 93, 99]
```

【例子】

```
a = [(i, j) for i in range(0, 3) for j in range(0, 3)]
print(a)

# [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

【例子】

```
x = [[i, j] for i in range(0, 3) for j in range(0, 3)]
print(x)
# [[0, 0], [0, 1], [0, 2], [1, 0], [1, 1], [1, 2], [2, 0], [2, 1], [2, 2]]

x[0][0] = 10
print(x)
# [[10, 0], [0, 1], [0, 2], [1, 0], [1, 1], [1, 2], [2, 0], [2, 1], [2, 2]]
```

【例子】

```
a = [(i, j) for i in range(0, 3) if i < 1 for j in range(0, 3) if j > 1]
print(a)

# [(0, 2)]
```

10.2 元组推导式

```
(expr for value in collection [if condition] )
```

【例子】

```
a = (x for x in range(10))
print(a)

# <generator object <genexpr> at 0x0000025BE511CC48>

print(tuple(a))

# (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
<generator object <genexpr> at 0x0000014CEC2E28B8>
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

10.3 字典推导式

```
{key_expr: value_expr for value in collection [if condition] }
```

【例子】

```
b = {i: i % 2 == 0 for i in range(10) if i % 3 == 0}
print(b)
# {0: True, 3: False, 6: True, 9: False}
```

10.4 集合推导式

```
{expr for value in collection [if condition] }
```

【例子】

```
c = {i for i in [1, 2, 3, 4, 5, 5, 6, 4, 3, 2, 1]}
print(c)
# {1, 2, 3, 4, 5, 6}
```

```
{1, 2, 3, 4, 5, 6}
```

其它

- `next(iterator[, default])` Return the next item from the iterator. If default is given

and the iterator is exhausted, it is returned instead of raising StopIteration.

【例子】

```
e = (i for i in range(10))
print(e)
# <generator object <genexpr> at 0x0000007A0B8D01B0>

print(next(e))  # 0
print(next(e))  # 1

for each in e:
    print(each, end=' ')

# 2 3 4 5 6 7 8 9
```

【例子】

```
s = sum([i for i in range(101)])
print(s)  # 5050
s = sum((i for i in range(101)))
print(s)  # 5050
```

11. 异常处理

异常就是运行期检测到的错误。

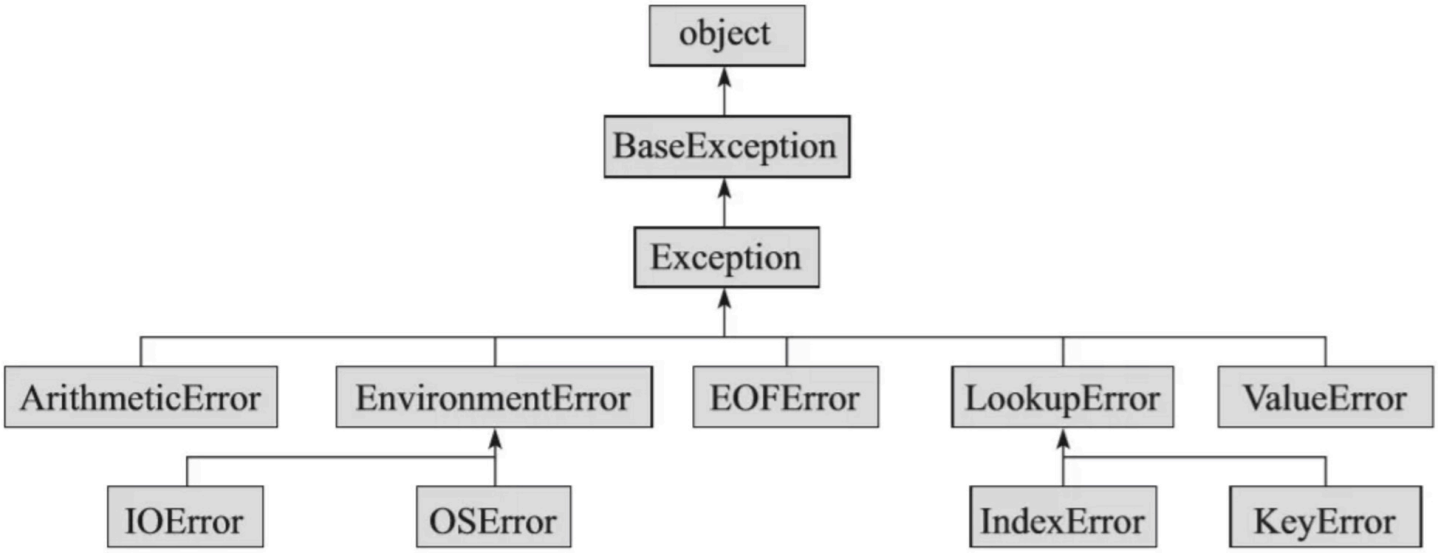
计算机语言针对可能出现的错误定义了异常类型，某种错误引发对应的异常时，异常处理程序将被启动，从而恢复程序的正常运行。

11.1 Python 标准异常总结

- BaseException：所有异常的 **基类**
- Exception：常规异常的 **基类**
- StandardError：所有的内建标准异常的基类
- ArithmeticError：所有数值计算异常的基类
- FloatingPointError：浮点计算异常
- OverflowError：数值运算超出最大限制
- ZeroDivisionError：除数为零
- AssertionError：断言语句(assert) 失败
- AttributeError：尝试访问未知的对象属性
- EOFError：没有内建输入，到达 EOF 标记

- EnvironmentError：操作系统异常的基类
- IOError：输入 / 输出操作失败
- OSError：操作系统产生的异常(例如打开一个不存在的文件)
- WindowsError：系统调用失败
- ImportError：导入模块失败的时候
- KeyboardInterrupt：用户中断执行
- LookupError：无效数据查询的基类
- IndexError：索引超出序列的范围
- KeyError：字典中查找一个不存在的关键字
- MemoryError：内存溢出(可通过删除对象释放内存)
- NameError：尝试访问一个不存在的变量
- UnboundLocalError：访问未初始化的本地变量
- ReferenceError：弱引用试图访问已经垃圾回收了的对象
- RuntimeError：一般的运行时异常
- NotImplementedError：尚未实现的方法
- SyntaxError：语法错误导致的异常
- IndentationError：缩进错误导致的异常
- TabError：Tab 和空格混用
- SystemError：一般的解释器系统异常
- TypeError：不同类型间的无效操作
- ValueError：传入无效的参数
- UnicodeError：Unicode 相关的异常
- UnicodeDecodeError：Unicode 解码时的异常
- UnicodeEncodeError：Unicode 编码错误导致的异常
- UnicodeTranslateError：Unicode 转换错误导致的异常

异常体系内部有层次关系，Python 异常体系中的部分关系如下所示：



11.2 Python 标准警告总结

- Warning：警告的基类
- DeprecationWarning：关于被弃用的特征的警告
- FutureWarning：关于构造将来语义会有改变的警告
- UserWarning：用户代码生成的警告
- PendingDeprecationWarning：关于特性将会被废弃的警告
- RuntimeWarning：可疑的运行时行为 (runtime behavior) 的警告
- SyntaxWarning：可疑语法的警告
- ImportWarning：用于在导入模块过程中触发的警告
- UnicodeWarning：与 Unicode 相关的警告
- BytesWarning：与字节或字节码相关的警告
- ResourceWarning：与资源使用相关的警告

11.3 try - except 语句

```
try:
    检测范围
except Exception[as reason]:
    出现异常后的处理代码
```

try 语句按照如下方式工作：

- 首先，执行 `try` 子句(在关键字 `try` 和关键字 `except` 之间的语句)
- 如果没有异常发生，忽略 `except` 子句， `try` 子句执行后结束。

- 如果在执行 `try` 子句的过程中发生了异常，那么 `try` 子句余下的部分将被忽略。如果异常的类型和 `except` 之后的名称相符，那么对应的 `except` 子句将被执行。最后执行 `try – except` 语句之后的代码。
- 如果一个异常没有与任何的 `except` 匹配，那么这个异常将会传递给上层的 `try` 中。

【例子】

```
try:
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError:
    print('打开文件出错')

# 打开文件出错
```

【例子】

```
try:
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError as error:
    print('打开文件出错 \n 原因是: ' + str(error))

# 打开文件出错
# 原因是: [Errno 2] No such file or directory: 'test.txt'
```

一个 `try` 语句可能包含多个 `except` 子句，分别来处理不同的特定的异常。最多只有一个分支会被执行。

【例子】

```
try:
    int("abc")
    s = 1 + '1'
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError as error:
    print('打开文件出错 \n 原因是: ' + str(error))
except TypeError as error:
    print('类型出错 \n 原因是: ' + str(error))
except ValueError as error:
    print('数值出错 \n 原因是: ' + str(error))

# 数值出错
# 原因是: invalid literal for int() with base 10: 'abc'
```

【例子】

```
dict1 = {'a': 1, 'b': 2, 'v': 22}
try:
    x = dict1['y']
except LookupError:
    print('查询错误')
except KeyError:
    print('键错误')
else:
    print(x)

# 查询错误
```

`try-except-else` 语句尝试查询不在 `dict` 中的键值对，从而引发了异常。这一异常准确地说应属于 `KeyError`，但由于 `KeyError` 是 `LookupError` 的子类，且将 `LookupError` 置于 `KeyError` 之前，因此程序优先执行该 `except` 代码块。所以，使用多个 `except` 代码块时，必须坚持对其规范排序，要从最具针对性的异常到最通用的异常。

【例子】

```
dict1 = {'a': 1, 'b': 2, 'v': 22}
try:
    x = dict1['y']
except KeyError:
    print('键错误')
except LookupError:
    print('查询错误')
else:
    print(x)

# 键错误
```

【例子】一个 `except` 子句可以同时处理多个异常，这些异常将被放在一个括号里成为一个元组。

```
try:
    s = 1 + '1'
    int("abc")
    f = open('test.txt')
    print(f.read())
    f.close()except (OSError, TypeError, ValueError) as error:
    print('出错了! \n 原因是: ' + str(error))

# 出错了!
# 原因是: unsupported operand type(s) for +: 'int' and 'str'
```

11.4 try - except - finally 语句

```
try:
    检测范围
except Exception[as reason]:
    出现异常后的处理代码
finally:
    无论如何都会被执行的代码
```

不管 `try` 子句里面有没有发生异常， `finally` 子句都会执行。

【例子】如果一个异常在 `try` 子句里被抛出，而又没有任何的 `except` 把它截住，那么这个异常会在 `finally` 子句执行后被抛出。

```
def divide(x, y):
    try:
        result = x / y
        print("result is", result)
    except ZeroDivisionError:
        print("division by zero!")
    finally:
        print("executing finally clause")


divide(2, 1)
# result is 2.0
# executing finally clause
divide(2, 0)
# division by zero!
# executing finally clause
divide("2", "1")
# executing finally clause
# TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

11.5 try - except - else 语句

如果在 `try` 子句执行时没有发生异常，Python 将执行 `else` 语句后的语句。

```
try:
    检测范围
except:
    出现异常后的处理代码
else:
    如果没有异常执行这块代码
```

使用 `except` 而不带任何异常类型，这不是一个很好的方式，我们不能通过该程序识别出具体的异

常信息，因为它捕获所有的异常。

```
try:
    检测范围
except(Exception1[, Exception2[,...ExceptionN]]]):
   发生以上多个异常中的一个，执行这块代码
else:
    如果没有异常执行这块代码
```

【例子】

```
try:
    fh = open("testfile.txt", "w")
    fh.write("这是一个测试文件，用于测试异常!!")
except IOError:
    print("Error: 没有找到文件或读取文件失败")
else:
    print("内容写入文件成功")
    fh.close()

# 内容写入文件成功
```

注意：`else` 语句的存在必须以 `except` 语句的存在为前提，在没有 `except` 语句的 `try` 语句中使用 `else` 语句，会引发语法错误。

11.6 raise 语句

Python 使用 `raise` 语句抛出一个指定的异常。

【例子】

```
try:
    raise NameError('HiThere')
except NameError:
    print('An exception flew by!')

# An exception flew by!
```